United States Patent Application Publication	20250285182
Kind Code	A1
Publication Date	September 11, 2025
Inventor(s)	Kavanagh; Kyle et al.

COMMUNICATIONS PROTOCOL BASED MESSAGE IDENTIFICATION TRANSMISSION

Abstract

A data transaction processing system receives electronic data transaction request messages from client computers over a data communication network and augments each message with hardware level data, and generates a monotonically increasing identification number for each electronic data transaction request message based on the hardware level data. The data transaction processing system transmits the identification number to the client computer utilizing transport layer protocols.

Inventors: **Kavanagh; Kyle (Chicago, IL), Acuña-Rohter; José Antonio (Des Plaines, IL)**

Applicant: **Chicago Mercantile Exchange Inc.** (Chicago, IL)

Family ID: **62063153**

Assignee: **Chicago Mercantile Exchange Inc. (Chicago, IL)**

Appl. No.: **19/212188**

Filed: **May 19, 2025**

Related U.S. Application Data

parent US continuation 18131625 20230406 parent-grant-document US 12321985 child US 19212188
parent US continuation 15470333 20170327 parent-grant-document US 11651428 child US 18131625

Publication Classification

Int. Cl.: **G06Q40/04** (20120101); **H04L5/00** (20060101); **H04L69/16** (20220101); **H04L69/163** (20220101); **H04L69/326** (20220101)

U.S. Cl.:

CPC **G06Q40/04** (20130101); **H04L5/0055** (20130101); **H04L69/16** (20130101); **H04L69/163** (20130101); **H04L69/326** (20130101);

Background/Summary

REFERENCE TO RELATED APPLICATIONS [0001] This application is a continuation under 37 C.F.R. § 1.53 (b) of U.S. patent application Ser. No. 18/131,625 filed Apr. 6, 2023, now U.S. Pat. No. _____, which is a continuation under 37 C.F.R. § 1.53 (b) of U.S. patent application Ser. No. 15/470,333 filed Mar. 27, 2017, now U.S. Pat. No. 11,651,428, all of which are incorporated by reference in their entirety.

BACKGROUND
[0002] A data transaction processing system receives electronic data transaction request messages specifying transactions to be performed. The data transaction processing system assigns an identification number for each requested transaction. The identification number is typically not generated until the electronic data transaction request message is being processed, and the identification number is not provided to the client computer that submitted the electronic data transaction request message until after the electronic data transaction request message has been processed. The processing speed of data transaction processing systems depends on the volume and the types of transactions being handled. The data transaction processing system may be configured to concurrently process a limited number of received transactions. Newly received transactions may have to wait or be stored in a queue before being processed if the data transaction processing system is busy processing another transaction. During times of heavy activity, many transactions may be queued before processing, increasing response time latency, the likelihood that the state of the system may change before a submitted transaction is actually processed, and user uncertainty and risk.
[0003] Depending on the state of the data transaction processing system and the requested transaction, the wait time to receive an identification number can become excessive. Moreover, in a low latency application, generating an identification number that complies with the data transaction processing system's rules can become a processing bottleneck.

Description

BRIEF DESCRIPTION OF THE DRAWINGS
[0004] FIG. **1** depicts an illustrative computer network system that may be used to implement aspects of the disclosed embodiments.
[0005] FIG. **2** depicts an illustrative embodiment of a general computer system for use with the disclosed embodiments.
[0006] FIG. **3**A depicts an illustrative embodiment of a data structure used to implement aspects of the disclosed embodiments.
[0007] FIG. **3**B depicts an illustrative embodiment of an alternative data structure used to implement aspects of the disclosed embodiments.
[0008] FIG. **3**C depicts an illustrative embodiment of an order book data structure used to implement aspects of the disclosed embodiments.
[0009] FIG. **4** illustrates an example match engine module in accordance with the disclosed embodiments.
[0010] FIG. **5** illustrates an example message management module in accordance with the

disclosed embodiments.
[0011] FIG. **6** illustrates an example transport layer electronic message data segment in accordance with the disclosed embodiments.
[0012] FIG. **7** illustrates an example system including client computers connected over TCP sessions to an example message management module in accordance with the disclosed embodiments.
[0013] FIG. **8** illustrates an example flowchart in accordance with the disclosed embodiments.
[0014] FIG. **9** illustrates an example messaging sequence timing diagram in accordance with the disclosed embodiments.
[0015] FIG. **10** illustrates another example messaging sequence timing diagram in accordance with the disclosed embodiments.
DETAILED DESCRIPTION
[0016] The disclosed embodiments relate generally to a data transaction processing system that receives electronic data transaction request messages from client computers including requests to perform transactions on data objects. The data transaction processing system generates an identification number for each electronic data transaction request message based on hardware located in the data transaction processing system and transmits the identification number back to the client computer before the electronic data transaction request message is processed by the data transaction processing system. The hardware may be a time-stamping device or a counter that assigns a monotonically increasing time or number, respectively, to incoming electronic message data segments.
[0017] The electronic data transaction request message is also communicated to a destination application, which may process, consume, or otherwise use the electronic message packet or electronic message data segment payload, and report on the results of the processing, consumption, and/or usage of the messages back to the origin or source of the message. Thus, the electronic data transaction request message is processed by the destination application.
[0018] Once a user has received an identification number for a previously submitted but not yet processed electronic data transaction request message (e.g., a first electronic data transaction request message), the user may reference the identification number in a second electronic data transaction request message to modify the previously submitted but not yet processed (e.g., by the destination application) first electronic data transaction request message. The disclosed embodiments improve on identification number generation and electronic data transaction request message control and modification. In one embodiment, the client computer (which submitted the electronic data transaction request message) is provided the identification number for the electronic data transaction request message before the electronic data transaction request message is processed by a destination application. The data transaction processing system may identify the destination application based on the transaction requested in the electronic data transaction request message.
[0019] Users of the data transaction processing system typically submit electronic data transaction request messages to implement a strategy. For example, when the data transaction processing system is implemented within a financial exchange computing system, the users may be traders who submit orders to buy or sell financial instruments at specific values. Traders' strategies may rapidly vary with the state of an electronic marketplace, and may accordingly necessitate submission of many additional electronic data transaction request messages, which may be related to (e.g., modify) previously submitted electronic data transaction request messages. The disclosed embodiments enable the data transaction processing system to generate identification numbers based on hardware timestamp information and transmit same to client computers via transport layer (e.g., TCP/IP) information. The disclosed embodiments also enable users to control, e.g., modify, or cancel, previously submitted electronic data transaction request messages substantially instantaneously, without needing to wait for the data transaction processing system to process the

previously submitted electronic data transaction request messages, thus increasing user control, flexibility and convenience. In many data transaction processing systems, processing wait time delays without any control over the corresponding electronic data transaction request messages can be an undesirable risk factor for the electronic data transaction request message submitter.

[0020] The disclosed data transaction processing system greatly reduces the processing, e.g., software processing, necessary to generate an identification number, such that the data transaction processing system can share each client computer's electronic data transaction request message's corresponding identification number with the client computer in real time/substantially instantly through transport layer protocols.

[0021] The identification numbers may be unique so that they differ from every other identification number ever created/used by the data transaction processing system. Or, the identification number may be unique compared to all the other identification numbers active in the system. Thus, “unique” may refer to substantially unique, such that each unique identification number allows distinguishing the electronic data transaction request message from any other electronic data transaction request message in the system, active or otherwise ever generated/used.

[0022] The data transaction processing system, may, in one embodiment, operate in a stateful manner, i.e., depend upon historical/prior messages received, and/or rely upon previous results thereof or previous decisions made, by the transaction processing system. The data transaction processing system may also access data structures storing information about a current environment state to determine if orders or messages match.

[0023] The disclosed embodiments also improve upon the technical field of networking by integrating hardware level/timecard information into transport layer protocol messaging, which enables more control by client computers over previously submitted messages. The disclosed embodiments also improve on the field of data processing by eliminating a separate software based identification number generation module, and instead using hardware based information for identification number generation. The disclosed system is a specific implementation and practical application of a hardware based device that timestamps incoming electronic data transaction request messages, generates a unique, monotonically increasing identification number and transmits same instantly using transport layer messaging.

[0024] At least some of the problems solved by the disclosed encoding system are specifically rooted in technology, where transaction processing of a large volume of messages is a time-consuming, resource intensive task that requires unique identification number generation, and the time required to generate and transmit identification numbers is a processing bottleneck for several subsequent workflows, and are solved by means of a technical solution, namely, generating identification numbers based on hardware level time data augmentation and transmitting same using TCP layer messaging, allowing client computers far greater control over messages transmitted to the data transaction processing system. The disclosed embodiments solve a communications network-centric problem of sending large amounts of electronic data transaction request messages to a data transaction processing system whose processing loads, and thus responsiveness to the electronic data transaction request messages, greatly varies over time. Accordingly, the resulting problem is a problem arising in computer systems due to client computers submitting a high volume of messages to a data transaction processing system. The solutions disclosed herein are, in one embodiment, implemented as automatic responses and actions by an exchange computing system computer.

[0025] The disclosed embodiments may be directed to an exchange computing system that includes multiple hardware matching processors that match, or attempt to match, electronic data transaction request messages with other electronic data transaction request messages counter thereto. Incoming electronic data transaction request messages may be received from different client computers over a data communication network, and output electronic data transaction result messages may be transmitted to the client computers and may be indicative of results of the attempts to match

incoming electronic data transaction request messages.
[0026] The disclosed embodiments may be implemented in a data transaction processing system that processes data items or objects. Customer or user devices (e.g., computers) may submit electronic data transaction request messages, e.g., inbound messages, to the data transaction processing system over a data communication network. The electronic data transaction request messages may include, for example, transaction matching parameters, such as instructions and/or values, for processing the data transaction request messages within the data transaction processing system. The instructions may be to perform transactions, e.g., buy or sell a quantity of a product at a range of values defined equations. Products, e.g., financial instruments, or order books representing the state of an electronic marketplace for a product, may be represented as data objects within the exchange computing system. The instructions may also be conditional, e.g., buy or sell a quantity of a product at a given value if a trade for the product is executed at some other reference value. The data transaction processing system may include various specifically configured matching processors that match, e.g., automatically, electronic data transaction request messages for the same one of the data items or objects. The specifically configured matching processors may match, or attempt to match, electronic data transaction request messages based on multiple transaction matching parameters from the different client computers. The specifically configured matching processors may additionally generate information indicative of a state of an environment (e.g., the state of the order book) based on the processing, and report this information to data recipient computing systems via outbound messages published via one or more data feeds.
[0027] For example, one exemplary environment where the disclosed embodiments may be desirable is in financial markets, and in particular, electronic financial exchanges, such as a futures exchange, such as the Chicago Mercantile Exchange Inc. (CME).
Exchange Computing System
[0028] A financial instrument trading system, such as a futures exchange, such as the Chicago Mercantile Exchange Inc. (CME), provides a contract market where financial instruments, e.g., futures and options on futures, are traded using electronic systems. “Futures” is a term used to designate all contracts for the purchase or sale of financial instruments or physical commodities for future delivery or cash settlement on a commodity futures exchange. A futures contract is a legally binding agreement to buy or sell a commodity at a specified price at a predetermined future time. An option contract is the right, but not the obligation, to sell or buy the underlying instrument (in this case, a futures contract) at a specified price on or before a certain expiration date. An option contract offers an opportunity to take advantage of futures price moves without actually having a futures position. The commodity to be delivered in fulfillment of the contract, or alternatively the commodity for which the cash market price shall determine the final settlement price of the futures contract, is known as the contract's underlying reference or “underlier.” The underlying or underlier for an options contract is the corresponding futures contract that is purchased or sold upon the exercise of the option.
[0029] The terms and conditions of each futures contract are standardized as to the specification of the contract's underlying reference commodity, the quality of such commodity, quantity, delivery date, and means of contract settlement. Cash settlement is a method of settling a futures contract whereby the parties effect final settlement when the contract expires by paying/receiving the loss/gain related to the contract in cash, rather than by effecting physical sale and purchase of the underlying reference commodity at a price determined by the futures contract, price. Options and futures may be based on more generalized market indicators, such as stock indices, interest rates, futures contracts and other derivatives.
[0030] An exchange may provide for a centralized “clearing house” through which trades made must be confirmed, matched, and settled each day until offset or delivered. The clearing house may be an adjunct to an exchange, and may be an operating division of an exchange, which is responsible for settling trading accounts, clearing trades, collecting and maintaining performance

bond funds, regulating delivery, and reporting trading data. One of the roles of the clearing house is to mitigate credit risk. Clearing is the procedure through which the clearing house becomes buyer to each seller of a futures contract, and seller to each buyer, also referred to as a novation, and assumes responsibility for protecting buyers and sellers from financial loss due to breach of contract, by assuring performance on each contract. A clearing member is a firm qualified to clear trades through the clearing house.

[0031] An exchange computing system may operate under a central counterparty model, where the exchange acts as an intermediary between market participants for the transaction of financial instruments. In particular, the exchange computing system novates itself into the transactions between the market participants, i.e., splits a given transaction between the parties into two separate transactions where the exchange computing system substitutes itself as the counterparty to each of the parties for that part of the transaction, sometimes referred to as a novation. In this way, the exchange computing system acts as a guarantor and central counterparty and there is no need for the market participants to disclose their identities to each other, or subject themselves to credit or other investigations by a potential counterparty. For example, the exchange computing system insulates one market participant from the default by another market participant. Market participants need only meet the requirements of the exchange computing system. Anonymity among the market participants encourages a more liquid market environment as there are lower barriers to participation. The exchange computing system can accordingly offer benefits such as centralized and anonymous matching and clearing.

[0032] A match engine within a financial instrument trading system may comprise a transaction processing system that processes a high volume, e.g., millions, of messages or orders in one day. The messages are typically submitted from market participant computers. Exchange match engine systems may be subject to variable messaging loads due to variable market messaging activity. Performance of a match engine depends to a certain extent on the magnitude of the messaging load and the work needed to process that message at any given time. An exchange match engine may process large numbers of messages during times of high volume messaging activity. With limited processing capacity, high messaging volumes may increase the response time or latency experienced by market participants.

[0033] The disclosed embodiments recognize that electronic messages such as incoming messages from market participants, i.e., “outright” messages, e.g., trade order messages, etc., are sent from client devices associated with market participants, or their representatives, to an electronic trading or market system. For example, a market participant may submit an electronic message to the electronic trading system that includes an associated specific action to be undertaken by the electronic trading system, such as entering a new trade order into the market or modifying an existing order in the market. In one embodiment, if a participant wishes to modify a previously sent request, e.g., a prior order which has not yet been processed or traded, they may send a request message comprising a request to modify the prior request.

Electronic Data Transaction Request Messages

[0034] As used herein, a financial message, or an electronic message, refers both to messages communicated by market participants to an electronic trading or market system and vice versa. The messages may be communicated using packeting or other techniques operable to communicate information between systems and system components. Some messages may be associated with actions to be taken in the electronic trading or market system. In particular, in one embodiment, upon receipt of a request, a token is allocated and included in a TCP shallow acknowledgment transmission sent back to the participant acknowledging receipt of the request. It should be appreciated that while this shallow acknowledgment is, in some sense, a response to the request, it does not confirm the processing of an order included in the request. The participant, i.e., their device, then sends back a TCP acknowledgment which acknowledges receipt of the shallow acknowledgment and token.

[0035] Financial messages communicated to the electronic trading system, also referred to as "inbound" messages, may include associated actions that characterize the messages, such as trader orders, order modifications, order cancellations and the like, as well as other message types. Inbound messages may be sent from market participants, or their representatives, e.g., trade order messages, etc., to an electronic trading or market system. For example, a market participant may submit an electronic message to the electronic trading system that includes an associated specific action to be undertaken by the electronic trading system, such as entering a new trade order into the market or modifying an existing order in the market. In one exemplary embodiment, the incoming request itself, e.g., the inbound order entry, may be referred to as an iLink message. iLink is a bidirectional communications/message protocol/message format implemented by the Chicago Mercantile Exchange Inc.
[0036] Financial messages communicated from the electronic trading system, referred to as "outbound" messages, may include messages responsive to inbound messages, such as confirmation messages, or other messages such as market update messages, quote messages, and the like. Outbound messages may be disseminated via data feeds.
[0037] Financial messages may further be categorized as having or reflecting an impact on a market or electronic marketplace, also referred to as an "order book" or "book," for a traded product, such as a prevailing price therefore, number of resting orders at various price levels and quantities thereof, etc., or not having or reflecting an impact on a market or a subset or portion thereof. In one embodiment, an electronic order book may be understood to be an electronic collection of the outstanding or resting orders for a financial instrument.
[0038] For example, a request to place a trade may result in a response indicative of the trade either being matched with, or being rested on an order book to await, a suitable counter-order. This response may include a message directed solely to the trader who submitted the order to acknowledge receipt of the order and report whether it was matched, and the extent thereto, or rested. The response may further include a message to all market participants reporting a change in the order book due to the order. This response may take the form of a report of the specific change to the order book, e.g., an order for quantity X at price Y was added to the book (referred to, in one embodiment, as a Market By Order message), or may simply report the result, e.g., price level Y now has orders for a total quantity of Z (where Z is the sum of the previous resting quantity plus quantity X of the new order). In some cases, requests may elicit a non-impacting response, such as temporally proximate to the receipt of the request, and then cause a separate market-impact reflecting response at a later time. For example, a stop order, fill or kill order, also known as an immediate or cancel order, or other conditional request may not have an immediate market impacting effect, if at all, until the requisite conditions are met.
[0039] An acknowledgement or confirmation of receipt, e.g., a non-market impacting communication, may be sent to the trader simply confirming that the order was received. Upon the conditions being met and a market impacting result thereof occurring, a market-impacting message may be transmitted as described herein both directly back to the submitting market participant and to all market participants (in a Market By Price "MBP" e.g., Aggregated By Value ("ABV") book, or Market By Order "MBO", e.g., Per Order ("PO") book format). It should be appreciated that additional conditions may be specified, such as a time or price limit, which may cause the order to be dropped or otherwise canceled and that such an event may result in another non-market-impacting communication instead. In some implementations, market impacting communications may be communicated separately from non-market impacting communications, such as via a separate communications channel or feed.
[0040] It should be further appreciated that various types of market data feeds may be provided which reflect different markets or aspects thereof. Market participants may then, for example, subscribe to receive those feeds of interest to them. For example, data recipient computing systems may choose to receive one or more different feeds. As market impacting communications usually

tend to be more important to market participants than non-impacting communications, this separation may reduce congestion and/or noise among those communications having or reflecting an impact on a market or portion thereof. Furthermore, a particular market data feed may only communicate information related to the top buy/sell prices for a particular product, referred to as "top of book" feed, e.g., only changes to the top 10 price levels are communicated. Such limitations may be implemented to reduce consumption of bandwidth and message generation resources. In this case, while a request message may be considered market-impacting if it affects a price level other than the top buy/sell prices, it will not result in a message being sent to the market participants.

[0041] Examples of the various types of market data feeds which may be provided by electronic trading systems, such as the CME, in order to provide different types or subsets of market information or to provide such information in different formats include Market By Order or Per Order, Market Depth (also known as Market by Price or Aggregated By Value to a designated depth of the book), e.g., CME offers a 10-deep market by price feed, Top of Book (a single depth Market by Price feed), and combinations thereof. There may also be all manner of specialized feeds in terms of the content, i.e., providing, for example, derived data, such as a calculated index.

[0042] Market data feeds may be characterized as providing a "view" or "overview" of a given market, an aggregation or a portion thereof or changes thereto. For example, a market data feed, such as a Market By Price ("MBP") feed, also known as an Aggregated By Value ("ABV") feed, may convey, with each message, the entire/current state of a market, or portion thereof, for a particular product as a result of one or more market impacting events. For example, an MBP message may convey a total quantity of resting buy/sell orders at a particular price level in response to a new order being placed at that price. An MBP message may convey a quantity of an instrument which was traded in response to an incoming order being matched with one or more resting orders. MBP messages may only be generated for events affecting a portion of a market, e.g., only the top 10 resting buy/sell orders and, thereby, only provide a view of that portion. As used herein, a market impacting request may be said to impact the "view" of the market as presented via the market data feed.

[0043] An MBP feed may utilize different message formats for conveying different types of market impacting events. For example, when a new order is rested on the order book, an MBP message may reflect the current state of the price level to which the order was added, e.g., the new aggregate quantity and the new aggregate number of resting orders. As can be seen, such a message conveys no information about the individual resting orders, including the newly rested order, themselves to the market participants. Only the submitting market participant, who receives a separate private message acknowledging the event, knows that it was their order that was added to the book. Similarly, when a trade occurs, an MBP message may be sent which conveys the price at which the instrument was traded, the quantity traded and the number of participating orders, but may convey no information as to whose particular orders contributed to the trade. MBP feeds may further batch reporting of multiple events, i.e., report the result of multiple market impacting events in a single message.

[0044] Alternatively, a market data feed, referred to as a Market By Order ("MBO") feed also known as a Per Order ("PO") feed, may convey data reflecting a change that occurred to the order book rather than the result of that change, e.g., that order ABC for quantity X was added to price level Y or that order ABC and order XYZ traded a quantity X at a price Y. In this case, the MBO message identifies only the change that occurred so a market participant wishing to know the current state of the order book must maintain their own copy and apply the change reflected in the message to know the current state. As can be seen, MBO/PO messages may carry much more data than MBP/ABV messages because MBO/PO messages reflect information about each order, whereas MBP/ABV messages contain information about orders affecting some predetermined value levels. Furthermore, because specific orders, but not the submitting traders thereof, are

identified, other market participants may be able to follow that order as it progresses through the market, e.g., as it is modified, canceled, traded, etc.

[0045] An ABV book data object may include information about multiple values. The ABV book data object may be arranged and structured so that information about each value is aggregated together. Thus, for a given value V, the ABV book data object may aggregate all the information by value, such as for example, the number of orders having a certain position at value V, the quantity of total orders resting at value V, etc. Thus, the value field may be the key, or may be a unique field, within an ABV book data object. In one embodiment, the value for each entry within the ABV book data object is different. In one embodiment, information in an ABV book data object is presented in a manner such that the value field is the most granular field of information.

[0046] A PO book data object may include information about multiple orders. The PO book data object may be arranged and structured so that information about each order is represented. Thus, for a given order O, the PO book data object may provide all of the information for order O. Thus, the order field may be the key, or may be a unique field, within a PO book data object. In one embodiment, the order ID for each entry within the PO book data object is different. In one embodiment, information in a PO book data object is presented in a manner such that the order field is the most granular field of information.

[0047] Thus, the PO book data object may include data about unique orders, e.g., all unique resting orders for a product, and the ABV book data object may include data about unique values, e.g., up to a predetermined level, e.g., top ten price or value levels, for a product.

[0048] It should be appreciated that the number, type and manner of market data feeds provided by an electronic trading system are implementation dependent and may vary depending upon the types of products traded by the electronic trading system, customer/trader preferences, bandwidth and data processing limitations, etc. and that all such feeds, now available or later developed, are contemplated herein. MBP/ABV and MBO/PO feeds may refer to categories/variations of market data feeds, distinguished by whether they provide an indication of the current state of a market resulting from a market impacting event (MBP) or an indication of the change in the current state of a market due to a market impacting event (MBO).

[0049] Messages, whether MBO or MBP, generated responsive to market impacting events which are caused by a single order, such as a new order, an order cancellation, an order modification, etc., are fairly simple and compact and easily created and transmitted. However, messages, whether MBO or MBP, generated responsive to market impacting events which are caused by more than one order, such as a trade, may require the transmission of a significant amount of data to convey the requisite information to the market participants. For trades involving a large number of orders, e.g., a buy order for a quantity of 5000 which matches 5000 sell orders each for a quantity of 1, a significant amount of information may need to be sent, e.g., data indicative of each of the 5000 trades that have participated in the market impacting event.

[0050] In one embodiment, an exchange computing system may generate multiple order book objects, one for each type of view that is published or provided. For example, the system may generate a PO book object and an ABV book object. It should be appreciated that each book object, or view for a product or market, may be derived from the Per Order book object, which includes all the orders for a given financial product or market.

[0051] An inbound message may include an order that affects the PO book object, the ABV book object, or both. An outbound message may include data from one or more of the structures within the exchange computing system, e.g., the PO book object queues or the ABV book object queues.

[0052] Furthermore, each participating trader needs to receive a notification that their particular order has traded. Continuing with the example, this may require sending 5001 individual trade notification messages, or even 10,000+ messages where each contributing side (buy vs. sell) is separately reported, in addition to the notification sent to all of the market participants.

[0053] As detailed in U.S. patent application Ser. No. 14/100,788, the entirety of which is

incorporated by reference herein and relied upon, it may be recognized that trade notifications sent to all market participants may include redundant information repeated for each participating trade and a structure of an MBP trade notification message may be provided which results in a more efficient communication of the occurrence of a trade. The message structure may include a header portion which indicates the type of transaction which occurred, i.e., a trade, as well as other general information about the event, an instrument portion which comprises data about each instrument which was traded as part of the transaction, and an order portion which comprises data about each participating order. In one embodiment, the header portion may include a message type, Transaction Time, Match Event Indicator, and Number of Market Data Entries ("No. MD Entries") fields. The instrument portion may include a market data update action indicator ("MD Update Action"), an indication of the Market Data Entry Type ("MD Entry Type"), an identifier of the instrument/security involved in the transaction ("Security ID"), a report sequence indicator ("Rpt Seq"), the price at which the instrument was traded ("MD Entry PX"), the aggregate quantity traded at the indicated price ("ConsTradeQty"), the number of participating orders ("NumberOfOrders"), and an identifier of the aggressor side ("Aggressor Side") fields. The order portion may further include an identifier of the participating order ("Order ID"), described in more detail below, and the quantity of the order traded ("MD Entry Size") fields. It should be appreciated that the particular fields included in each portion are implementation dependent and that different fields in addition to, or in lieu of, those listed may be included depending upon the implementation. It should be appreciated that the exemplary fields can be compliant with the FIX binary and/or FIX/FAST protocol for the communication of the financial information.

[0054] The instrument portion contains a set of fields, e.g., seven fields accounting for 23 bytes, which are repeated for each participating instrument. In complex trades, such as trades involving combination orders or strategies, e.g., spreads, or implied trades, there may be multiple instruments being exchanged among the parties. In one embodiment, the order portion includes only one field, accounting for 4 bytes, for each participating order which indicates the quantity of that order which was traded. As will be discussed below, the order portion may further include an identifier of each order, accounting for an additional 8 bytes, in addition to the quantity thereof traded. As should be appreciated, data which would have been repeated for each participating order, is consolidated or otherwise summarized in the header and instrument portions of the message thereby eliminating redundant information and, overall, significantly reducing the size of the message.

[0055] While the disclosed embodiments will be discussed with respect to an MBP market data feed, it should be appreciated that the disclosed embodiments may also be applicable to an MBO market data feed.

Market Segment Gateway

[0056] In one embodiment, the disclosed system may include a Market Segment Gateway ("MSG") that is the point of ingress/entry and/or egress/departure for all transactions, i.e., the network traffic/packets containing the data therefore, specific to a single market at which the order of receipt of those transactions may be ascribed. An MSG or Market Segment Gateway may be utilized for the purpose of deterministic operation of the market. The electronic trading system may include multiple markets, and because the electronic trading system includes one MSG for each market/product implemented thereby, the electronic trading system may include multiple MSGs. For more detail on deterministic operation in a trading system, see U.S. patent application Ser. No. 14/074,667 entitled "Transactionally Deterministic High Speed Financial Exchange Having Improved, Efficiency, Communication, Customization, Performance, Access, Trading Opportunities, Credit Controls, And Fault Tolerance" and filed on Nov. 7, 2013, the entire disclosure of which is incorporated by reference herein and relied upon.

[0057] For example, a participant may send a request for a new transaction, e.g., a request for a new order, to the MSG. The MSG extracts or decodes the request message and determines the characteristics of the request message.

[0058] The MSG may include, or otherwise be coupled with, a buffer, cache, memory, database, content addressable memory, data store or other data storage mechanism, or combinations thereof, which stores data indicative of the characteristics of the request message. The request is passed to the transaction processing system, e.g., the match engine.

[0059] An MSG or Market Segment Gateway may be utilized for the purpose of deterministic operation of the market. Transactions for a particular market may be ultimately received at the electronic trading system via one or more points of entry, e.g., one or more communications interfaces, at which the disclosed embodiments apply determinism, which as described may be at the point where matching occurs, e.g., at each match engine (where there may be multiple match engines, each for a given product/market, or moved away from the point where matching occurs and closer to the point where the electronic trading system first becomes “aware” of the incoming transaction, such as the point where transaction messages, e.g., orders, ingress the electronic trading system. Generally, the terms “determinism” or “transactional determinism” may refer to the processing, or the appearance thereof, of orders in accordance with defined business rules. Accordingly, as used herein, the point of determinism may be the point at which the electronic trading system ascribes an ordering to incoming transactions/orders relative to other incoming transactions/orders such that the ordering may be factored into the subsequent processing, e.g., matching, of those transactions/orders as will be described. For more detail on deterministic operation in a trading system, see U.S. patent application Ser. No. 14/074,675, filed on Nov. 7, 2013, published as U.S. Patent Publication No. 2015/0127516 (“the '516 Publication”), entitled “Transactionally Deterministic High Speed Financial Exchange Having Improved, Efficiency, Communication, Customization, Performance, Access, Trading Opportunities, Credit Controls, And Fault Tolerance”, the entirety of which is incorporated by reference herein and relied upon.

Electronic Trading

[0060] Electronic trading of financial instruments, such as futures contracts, is conducted by market participants sending orders, such as to buy or sell one or more futures contracts, in electronic form to the exchange. These electronically submitted orders to buy and sell are then matched, if possible, by the exchange, i.e., by the exchange's matching engine, to execute a trade. Outstanding (unmatched, wholly unsatisfied/unfilled or partially satisfied/filled) orders are maintained in one or more data structures or databases referred to as “order books,” such orders being referred to as “resting,” and made visible, i.e., their availability for trading is advertised, to the market participants through electronic notifications/broadcasts, referred to as market data feeds. An order book is typically maintained for each product, e.g., instrument, traded on the electronic trading system and generally defines or otherwise represents the state of the market for that product, i.e., the current prices at which the market participants are willing buy or sell that product. As such, as used herein, an order book for a product may also be referred to as a market for that product.

[0061] Upon receipt of an incoming order to trade in a particular financial instrument, whether for a single-component financial instrument, e.g., a single futures contract, or for a multiple-component financial instrument, e.g., a combination contract such as a spread contract, a match engine, as described herein, will attempt to identify a previously received but unsatisfied order counter thereto, i.e., for the opposite transaction (buy or sell) in the same financial instrument at the same or better price (but not necessarily for the same quantity unless, for example, either order specifies a condition that it must be entirely filled or not at all).

[0062] Previously received but unsatisfied orders, i.e., orders which either did not match with a counter order when they were received or their quantity was only partially satisfied, referred to as a partial fill, are maintained by the electronic trading system in an order book database/data structure to await the subsequent arrival of matching orders or the occurrence of other conditions which may cause the order to be modified or otherwise removed from the order book.

[0063] If the match engine identifies one or more suitable previously received but unsatisfied

counter orders, they, and the incoming order, are matched to execute a trade there between to at least partially satisfy the quantities of one or both the incoming order or the identified orders. If there remains any residual unsatisfied quantity of the identified one or more orders, those orders are left on the order book with their remaining quantity to await a subsequent suitable counter order, i.e., to rest. If the match engine does not identify a suitable previously received but unsatisfied counter order, or the one or more identified suitable previously received but unsatisfied counter orders are for a lesser quantity than the incoming order, the incoming order is placed on the order book, referred to as “resting”, with original or remaining unsatisfied quantity, to await a subsequently received suitable order counter thereto. The match engine then generates match event data reflecting the result of this matching process. Other components of the electronic trading system, as will be described, then generate the respective order acknowledgment and market data messages and transmit those messages to the market participants.

[0064] Matching, which is a function typically performed by the exchange, is a process, for a given order which specifies a desire to buy or sell a quantity of a particular instrument at a particular price, of seeking/identifying one or more wholly or partially, with respect to quantity, satisfying counter orders thereto, e.g., a sell counter to an order to buy, or vice versa, for the same instrument at the same, or sometimes better, price (but not necessarily the same quantity), which are then paired for execution to complete a trade between the respective market participants (via the exchange) and at least partially satisfy the desired quantity of one or both of the order and/or the counter order, with any residual unsatisfied quantity left to await another suitable counter order, referred to as “resting.” A match event may occur, for example, when an aggressing order matches with a resting order. In one embodiment, two orders match because one order includes instructions for or specifies buying a quantity of a particular instrument at a particular price, and the other order includes instructions for or specifies selling a (different or same) quantity of the instrument at a same or better price. It should be appreciated that performing an instruction associated with a message may include attempting to perform the instruction. Whether or not an exchange computing system is able to successfully perform an instruction may depend on the state of the electronic marketplace.

[0065] While the disclosed embodiments will be described with respect to a product by product or market by market implementation, e.g. implemented for each market/order book, it will be appreciated that the disclosed embodiments may be implemented so as to apply across markets for multiple products traded on one or more electronic trading systems, such as by monitoring an aggregate, correlated or other derivation of the relevant indicative parameters as described herein.

[0066] While the disclosed embodiments may be discussed in relation to futures and/or options on futures trading, it should be appreciated that the disclosed embodiments may be applicable to any equity, fixed income security, currency, commodity, options or futures trading system or market now available or later developed. It may be appreciated that a trading environment, such as a futures exchange as described herein, implements one or more economic markets where rights and obligations may be traded. As such, a trading environment may be characterized by a need to maintain market integrity, transparency, predictability, fair/equitable access and participant expectations with respect thereto. In addition, it may be appreciated that electronic trading systems further impose additional expectations and demands by market participants as to transaction processing speed, latency, capacity and response time, while creating additional complexities relating thereto. Accordingly, as will be described, the disclosed embodiments may further include functionality to ensure that the expectations of market participants are met, e.g., that transactional integrity and predictable system responses are maintained.

[0067] Financial instrument trading systems allow traders to submit orders and receive confirmations, market data, and other information electronically via electronic messages exchanged using a network. Electronic trading systems ideally attempt to offer a more efficient, fair and balanced market where market prices reflect a true consensus of the value of traded products

among the market participants, where the intentional or unintentional influence of any one market participant is minimized if not eliminated, and where unfair or inequitable advantages with respect to information access are minimized if not eliminated.

[0068] Electronic marketplaces attempt to achieve these goals by using electronic messages to communicate actions and related data of the electronic marketplace between market participants, clearing firms, clearing houses, and other parties. The messages can be received using an electronic trading system, wherein an action or transaction associated with the messages may be executed. For example, the message may contain information relating to an order to buy or sell a product in a particular electronic marketplace, and the action associated with the message may indicate that the order is to be placed in the electronic marketplace such that other orders which were previously placed may potentially be matched to the order of the received message. Thus the electronic marketplace may conduct market activities through electronic systems.

Clearing House

[0069] The clearing house of an exchange clears, settles and guarantees matched transactions in contracts occurring through the facilities of the exchange. In addition, the clearing house establishes and monitors financial requirements for clearing members and conveys certain clearing privileges in conjunction with the relevant exchange markets.

[0070] The clearing house establishes clearing level performance bonds (margins) for all products of the exchange and establishes minimum performance bond requirements for customers of such products. A performance bond, also referred to as a margin requirement, corresponds with the funds that must be deposited by a customer with his or her broker, by a broker with a clearing member or by a clearing member with the clearing house, for the purpose of insuring the broker or clearing house against loss on open futures or options contracts. This is not a part payment on a purchase. The performance bond helps to ensure the financial integrity of brokers, clearing members and the exchange as a whole. The performance bond refers to the minimum dollar deposit required by the clearing house from clearing members in accordance with their positions. Maintenance, or maintenance margin, refers to a sum, usually smaller than the initial performance bond, which must remain on deposit in the customer's account for any position at all times. The initial margin is the total amount of margin per contract required by the broker when a futures position is opened. A drop in funds below this level requires a deposit back to the initial margin levels, i.e., a performance bond call. If a customer's equity in any futures position drops to or under the maintenance level because of adverse price action, the broker must issue a performance bond/margin call to restore the customer's equity. A performance bond call, also referred to as a margin call, is a demand for additional funds to bring the customer's account back up to the initial performance bond level whenever adverse price movements cause the account to go below the maintenance.

[0071] The exchange derives its financial stability in large part by removing debt obligations among market participants as they occur. This is accomplished by determining a settlement price at the close of the market each day for each contract and marking all open positions to that price, referred to as “mark to market.” Every contract is debited or credited based on that trading session's gains or losses. As prices move for or against a position, funds flow into and out of the trading account. In the case of the CME, each business day by 6:40 a.m. Chicago time, based on the mark-to-the-market of all open positions to the previous trading day's settlement price, the clearing house pays to or collects cash from each clearing member. This cash flow, known as settlement variation, is performed by CME's settlement banks based on instructions issued by the clearing house. All payments to and collections from clearing members are made in “same-day” funds. In addition to the 6:40 a.m. settlement, a daily intra-day mark-to-the market of all open positions, including trades executed during the overnight GLOBEX®, the CME's electronic trading systems, trading session and the current day's trades matched before 11:15 a.m., is performed using current prices. The resulting cash payments are made intra-day for same day value. In times of extreme price volatility,

the clearing house has the authority to perform additional intra-day mark-to-the-market calculations on open positions and to call for immediate payment of settlement variation. CME's mark-to-the-market settlement system differs from the settlement systems implemented by many other financial markets, including the interbank, Treasury securities, over-the-counter foreign exchange and debt, options, and equities markets, where participants regularly assume credit exposure to each other. In those markets, the failure of one participant can have a ripple effect on the solvency of the other participants. Conversely, CME's mark-to-the-market system does not allow losses to accumulate over time or allow a market participant the opportunity to defer losses associated with market positions.

[0072] While the disclosed embodiments may be described in reference to the CME, it should be appreciated that these embodiments are applicable to any exchange. Such other exchanges may include a clearing house that, like the CME clearing house, clears, settles and guarantees all matched transactions in contracts of the exchange occurring through its facilities. In addition, such clearing houses establish and monitor financial requirements for clearing members and convey certain clearing privileges in conjunction with the relevant exchange markets.

[0073] The disclosed embodiments are also not limited to uses by a clearing house or exchange for purposes of enforcing a performance bond or margin requirement. For example, a market participant may use the disclosed embodiments in a simulation or other analysis of a portfolio. In such cases, the settlement price may be useful as an indication of a value at risk and/or cash flow obligation rather than a performance bond. The disclosed embodiments may also be used by market participants or other entities to forecast or predict the effects of a prospective position on the margin requirement of the market participant.

Trading Environment

[0074] The embodiments may be described in terms of a distributed computing system. The particular examples identify a specific set of components useful in a futures and options exchange. However, many of the components and inventive features are readily adapted to other electronic trading environments. The specific examples described herein may teach specific protocols and/or interfaces, although it should be understood that the principles involved may be extended to, or applied in, other protocols and interfaces.

[0075] It should be appreciated that the plurality of entities utilizing or involved with the disclosed embodiments, e.g., the market participants, may be referred to by other nomenclature reflecting the role that the particular entity is performing with respect to the disclosed embodiments and that a given entity may perform more than one role depending upon the implementation and the nature of the particular transaction being undertaken, as well as the entity's contractual and/or legal relationship with another market participant and/or the exchange.

[0076] An exemplary trading network environment for implementing trading systems and methods is shown in FIG. **1**. An exchange computer system **100** receives messages that include orders and transmits market data related to orders and trades to users, such as via wide area network **162** and/or local area network **160** and computer devices **150**, **152**, **154**, **156** and **158**, as described herein, coupled with the exchange computer system **100**.

[0077] Herein, the phrase “coupled with” is defined to mean directly connected to or indirectly connected through one or more intermediate components. Such intermediate components may include both hardware and software based components. Further, to clarify the use in the pending claims and to hereby provide notice to the public, the phrases “at least one of <A>, <B>, . . . and <N>” or “at least one of <A>, <B>, . . . <N>, or combinations thereof” are defined by the Applicant in the broadest sense, superseding any other implied definitions herebefore or hereinafter unless expressly asserted by the Applicant to the contrary, to mean one or more elements selected from the group comprising A, B, . . . and N, that is to say, any combination of one or more of the elements A, B, . . . or N including any one element alone or in combination with one or more of the other elements which may also include, in combination, additional elements not listed.

[0078] The exchange computer system **100** may be implemented with one or more mainframe, desktop or other computers, such as the example computer **200** described herein with respect to FIG. **2**. A user database **102** may be provided which includes information identifying traders and other users of exchange computer system **100**, such as account numbers or identifiers, user names and passwords. An account data module **104** may be provided which may process account information that may be used during trades.
[0079] A match engine module **106** may be included to match bid and offer prices and may be implemented with software that executes one or more algorithms for matching bids and offers. A trade database **108** may be included to store information identifying trades and descriptions of trades. In particular, a trade database may store information identifying the time that a trade took place and the contract price. An order book module **110** may be included to compute or otherwise determine current bid and offer prices, e.g., in a continuous auction market, or also operate as an order accumulation buffer for a batch auction market.
[0080] A market data module **112** may be included to collect market data and prepare the data for transmission to users.
[0081] A risk management module **114** may be included to compute and determine a user's risk utilization in relation to the user's defined risk thresholds. The risk management module **114** may also be configured to determine risk assessments or exposure levels in connection with positions held by a market participant. The risk management module **114** may be configured to administer, manage or maintain one or more margining mechanisms implemented by the exchange computer system **100**. Such administration, management or maintenance may include managing a number of database records reflective of margin accounts of the market participants. In some embodiments, the risk management module **114** implements one or more aspects of the disclosed embodiments, including, for instance, principal component analysis (PCA) based margining, in connection with interest rate swap (IRS) portfolios, as described herein.
[0082] A message management module **116** may be included to, among other things, receive, and extract orders from, electronic data transaction request messages. The message management module **116** may define a point of ingress into the exchange computer system **100** where messages are ordered and considered to be received by the system. This may be considered a point of determinism in the exchange computer system **100** that defines the earliest point where the system can ascribe an order of receipt to arriving messages. The point of determinism may or may not be at or near the demarcation point between the exchange computer system **100** and a public/internet network infrastructure. The message management module **116** processes messages by interpreting the contents of a message based on the message transmit protocol, such as the transmission control protocol (“TCP”), to provide the content of the message for further processing by the exchange computer system.
[0083] The message management module **116** may also be configured to detect characteristics of an order for a transaction to be undertaken in an electronic marketplace. For example, the message management module **116** may identify and extract order content such as a price, product, volume, and associated market participant for an order. The message management module **116** may also identify and extract data indicating an action to be executed by the exchange computer system **100** with respect to the extracted order. For example, the message management module **116** may determine the transaction type of the transaction requested in a given message. A message may include an instruction to perform a type of transaction. The transaction type may be, in one embodiment, a request/offer/order to either buy or sell a specified quantity or units of a financial instrument at a specified price or value. The message management module **116** may also identify and extract other order information and other actions associated with the extracted order. All extracted order characteristics, other information, and associated actions extracted from a message for an order may be collectively considered an order as described and referenced herein.
[0084] Order or message characteristics may include, for example, the state of the system after a

message is received, arrival time (e.g., the time a message arrives at the MSG or Market Segment Gateway), message type (e.g., new, modify, cancel), and the number of matches generated by a message. Order or message characteristics may also include market participant side (e.g., buyer or seller) or time in force (e.g., a good until end of day order that is good for the full trading day, a good until canceled ordered that rests on the order book until matched, or a fill or kill order that is canceled if not filled immediately).

[0085] An order processing module **118** may be included to decompose delta-based, spread instrument, bulk and other types of composite orders for processing by the order book module **110** and/or the match engine module **106**. The order processing module **118** may also be used to implement one or more procedures related to clearing an order. The order may be communicated from the message management module **118** to the order processing module **118**. The order processing module **118** may be configured to interpret the communicated order, and manage the order characteristics, other information, and associated actions as they are processed through an order book module **110** and eventually transacted on an electronic market. For example, the order processing module **118** may store the order characteristics and other content and execute the associated actions. In an embodiment, the order processing module may execute an associated action of placing the order into an order book for an electronic trading system managed by the order book module **110**. In an embodiment, placing an order into an order book and/or into an electronic trading system may be considered a primary action for an order. The order processing module **118** may be configured in various arrangements, and may be configured as part of the order book module **110**, part of the message management module **118**, or as an independent functioning module.

[0086] As an intermediary to electronic trading transactions, the exchange bears a certain amount of risk in each transaction that takes place. To that end, the clearing house implements risk management mechanisms to protect the exchange. One or more of the modules of the exchange computer system **100** may be configured to determine settlement prices for constituent contracts, such as deferred month contracts, of spread instruments, such as for example, settlement module **120**. A settlement module **120** (or settlement processor or other payment processor) may be included to provide one or more functions related to settling or otherwise administering transactions cleared by the exchange. Settlement module **120** of the exchange computer system **100** may implement one or more settlement price determination techniques. Settlement-related functions need not be limited to actions or events occurring at the end of a contract term. For instance, in some embodiments, settlement-related functions may include or involve daily or other mark to market settlements for margining purposes. In some cases, the settlement module **120** may be configured to communicate with the trade database **108** (or the memory (ies) on which the trade database **108** is stored) and/or to determine a payment amount based on a spot price, the price of the futures contract or other financial instrument, or other price data, at various times. The determination may be made at one or more points in time during the term of the financial instrument in connection with a margining mechanism. For example, the settlement module **120** may be used to determine a mark to market amount on a daily basis during the term of the financial instrument. Such determinations may also be made on a settlement date for the financial instrument for the purposes of final settlement.

[0087] In some embodiments, the settlement module **120** may be integrated to any desired extent with one or more of the other modules or processors of the exchange computer system **100**. For example, the settlement module **120** and the risk management module **114** may be integrated to any desired extent. In some cases, one or more margining procedures or other aspects of the margining mechanism(s) may be implemented by the settlement module **120**.

[0088] One or more of the above-described modules of the exchange computer system **100** may be used to gather or obtain data to support the settlement price determination, as well as a subsequent margin requirement determination. For example, the order book module **110** and/or the market data

module **112** may be used to receive, access, or otherwise obtain market data, such as bid-offer values of orders currently on the order books. The trade database **108** may be used to receive, access, or otherwise obtain trade data indicative of the prices and volumes of trades that were recently executed in a number of markets. In some cases, transaction data (and/or bid/ask data) may be gathered or obtained from open outcry pits and/or other sources and incorporated into the trade and market data from the electronic trading system(s).

[0089] It should be appreciated that concurrent processing limits may be defined by or imposed separately or in combination on one or more of the trading system components, including the user database **102**, the account data module **104**, the match engine module **106**, the trade database **108**, the order book module **110**, the market data module **112**, the risk management module **114**, the message management module **116**, the order processing module **118**, the settlement module **120**, or other component of the exchange computer system **100**.

[0090] The disclosed mechanisms may be implemented at any logical and/or physical point(s), or combinations thereof, at which the relevant information/data (e.g., message traffic and responses thereto) may be monitored or flows or is otherwise accessible or measurable, including one or more gateway devices, modems, the computers or terminals of one or more market participants, e.g., client computers, etc.

[0091] One skilled in the art will appreciate that one or more modules described herein may be implemented using, among other things, a tangible computer-readable medium comprising computer-executable instructions (e.g., executable software code). Alternatively, modules may be implemented as software code, firmware code, specifically configured hardware or processors, and/or a combination of the aforementioned. For example, the modules may be embodied as part of an exchange **100** for financial instruments. It should be appreciated the disclosed embodiments may be implemented as a different or separate module of the exchange computer system **100**, or a separate computer system coupled with the exchange computer system **100** so as to have access to margin account record, pricing, and/or other data. As described herein, the disclosed embodiments may be implemented as a centrally accessible system or as a distributed system, e.g., where some of the disclosed functions are performed by the computer systems of the market participants.

[0092] The trading network environment shown in FIG. **1** includes exemplary computer devices **150**, **152**, **154**, **156** and **158** which depict different exemplary methods or media by which a computer device may be coupled with the exchange computer system **100** or by which a user may communicate, e.g., send and receive, trade or other information therewith. It should be appreciated that the types of computer devices deployed by traders and the methods and media by which they communicate with the exchange computer system **100** is implementation dependent and may vary and that not all of the depicted computer devices and/or means/media of communication may be used and that other computer devices and/or means/media of communications, now available or later developed may be used. Each computer device, which may comprise a computer **200** described in more detail with respect to FIG. **2**, may include a central processor, specifically configured or otherwise, that controls the overall operation of the computer and a system bus that connects the central processor to one or more conventional components, such as a network card or modem. Each computer device may also include a variety of interface units and drives for reading and writing data or files and communicating with other computer devices and with the exchange computer system **100**. Depending on the type of computer device, a user can interact with the computer with a keyboard, pointing device, microphone, pen device or other input device now available or later developed.

[0093] An exemplary computer device **150** is shown directly connected to exchange computer system **100**, such as via a T1 line, a common local area network (LAN) or other wired and/or wireless medium for connecting computer devices, such as the network **220** shown in FIG. **2** and described with respect thereto. The exemplary computer device **150** is further shown connected to a radio **168**. The user of radio **168**, which may include a cellular telephone, smart phone, or other

wireless proprietary and/or non-proprietary device, may be a trader or exchange employee. The radio user may transmit orders or other information to the exemplary computer device **150** or a user thereof. The user of the exemplary computer device **150**, or the exemplary computer device **150** alone and/or autonomously, may then transmit the trade or other information to the exchange computer system **100**.

[0094] Exemplary computer devices **152** and **154** are coupled with a local area network ("LAN") **160** which may be configured in one or more of the well-known LAN topologies, e.g., star, daisy chain, etc., and may use a variety of different protocols, such as Ethernet, TCP/IP, etc. The exemplary computer devices **152** and **154** may communicate with each other and with other computer and other devices which are coupled with the LAN **160**. Computer and other devices may be coupled with the LAN **160** via twisted pair wires, coaxial cable, fiber optics or other wired or wireless media. As shown in FIG. **1**, an exemplary wireless personal digital assistant device ("PDA") **158**, such as a mobile telephone, tablet based compute device, or other wireless device, may communicate with the LAN **160** and/or the Internet **162** via radio waves, such as via WiFi, Bluetooth and/or a cellular telephone based data communications protocol. PDA **158** may also communicate with exchange computer system **100** via a conventional wireless hub **164**.

[0095] FIG. **1** also shows the LAN **160** coupled with a wide area network ("WAN") **162** which may be comprised of one or more public or private wired or wireless networks. In one embodiment, the WAN **162** includes the Internet **162**. The LAN **160** may include a router to connect LAN **160** to the Internet **162**. Exemplary computer device **156** is shown coupled directly to the Internet **162**, such as via a modem, DSL line, satellite dish or any other device for connecting a computer device to the Internet **162** via a service provider therefore as is known. LAN **160** and/or WAN **162** may be the same as the network **220** shown in FIG. **2** and described with respect thereto.

[0096] Users of the exchange computer system **100** may include one or more market makers **166** which may maintain a market by providing constant bid and offer prices for a derivative or security to the exchange computer system **100**, such as via one of the exemplary computer devices depicted. The exchange computer system **100** may also exchange information with other match or trade engines, such as trade engine **170**. One skilled in the art will appreciate that numerous additional computers and systems may be coupled to exchange computer system **100**. Such computers and systems may include clearing, regulatory and fee systems.

[0097] The operations of computer devices and systems shown in FIG. **1** may be controlled by computer-executable instructions stored on a non-transitory computer-readable medium. For example, the exemplary computer device **152** may store computer-executable instructions for receiving order information from a user, transmitting that order information to exchange computer system **100** in electronic messages, extracting the order information from the electronic messages, executing actions relating to the messages, and/or calculating values from characteristics of the extracted order to facilitate matching orders and executing trades. In another example, the exemplary computer device **154** may include computer-executable instructions for receiving market data from exchange computer system **100** and displaying that information to a user.

[0098] Numerous additional servers, computers, handheld devices, personal digital assistants, telephones and other devices may also be connected to exchange computer system **100**. Moreover, one skilled in the art will appreciate that the topology shown in FIG. **1** is merely an example and that the components shown in FIG. **1** may include other components not shown and be connected by numerous alternative topologies.

[0099] Referring now to FIG. **2**, an illustrative embodiment of a general computer system **200** is shown. The computer system **200** can include a set of instructions that can be executed to cause the computer system **200** to perform any one or more of the methods or computer based functions disclosed herein. The computer system **200** may operate as a standalone device or may be connected, e.g., using a network, to other computer systems or peripheral devices. Any of the components discussed herein, such as the processor **202**, may be a computer system **200** or a

component in the computer system **200**. The computer system **200** may be specifically configured to implement a match engine, margin processing, payment or clearing function on behalf of an exchange, such as the Chicago Mercantile Exchange, of which the disclosed embodiments are a component thereof.

[0100] In a networked deployment, the computer system **200** may operate in the capacity of a server or as a client user computer in a client-server user network environment, or as a peer computer system in a peer-to-peer (or distributed) network environment. The computer system **200** can also be implemented as or incorporated into various devices, such as a personal computer (PC), a tablet PC, a set-top box (STB), a personal digital assistant (PDA), a mobile device, a palmtop computer, a laptop computer, a desktop computer, a communications device, a wireless telephone, a land-line telephone, a control system, a camera, a scanner, a facsimile machine, a printer, a pager, a personal trusted device, a web appliance, a network router, switch or bridge, or any other machine capable of executing a set of instructions (sequential or otherwise) that specify actions to be taken by that machine. In a particular embodiment, the computer system **200** can be implemented using electronic devices that provide voice, video or data communication. Further, while a single computer system **200** is illustrated, the term "system" shall also be taken to include any collection of systems or sub-systems that individually or jointly execute a set, or multiple sets, of instructions to perform one or more computer functions.

[0101] As illustrated in FIG. **2**, the computer system **200** may include a processor **202**, e.g., a central processing unit (CPU), a graphics processing unit (GPU), or both. The processor **202** may be a component in a variety of systems. For example, the processor **202** may be part of a standard personal computer or a workstation. The processor **202** may be one or more general processors, digital signal processors, specifically configured processors, application specific integrated circuits, field programmable gate arrays, servers, networks, digital circuits, analog circuits, combinations thereof, or other now known or later developed devices for analyzing and processing data. The processor **202** may implement a software program, such as code generated manually (i.e., programmed).

[0102] The computer system **200** may include a memory **204** that can communicate via a bus **208**. The memory **204** may be a main memory, a static memory, or a dynamic memory. The memory **204** may include, but is not limited to, computer readable storage media such as various types of volatile and non-volatile storage media, including but not limited to random access memory, read-only memory, programmable read-only memory, electrically programmable read-only memory, electrically crasable read-only memory, flash memory, magnetic tape or disk, optical media and the like. In one embodiment, the memory **204** includes a cache or random access memory for the processor **202**. In alternative embodiments, the memory **204** is separate from the processor **202**, such as a cache memory of a processor, the system memory, or other memory. The memory **204** may be an external storage device or database for storing data. Examples include a hard drive, compact disc ("CD"), digital video disc ("DVD"), memory card, memory stick, floppy disc, universal serial bus ("USB") memory device, or any other device operative to store data. The memory **204** is operable to store instructions executable by the processor **202**. The functions, acts or tasks illustrated in the figures or described herein may be performed by the programmed processor **202** executing the instructions **212** stored in the memory **204**. The functions, acts or tasks are independent of the particular type of instructions set, storage media, processor or processing strategy and may be performed by software, hardware, integrated circuits, firm-ware, micro-code and the like, operating alone or in combination. Likewise, processing strategies may include multiprocessing, multitasking, parallel processing and the like.

[0103] As shown, the computer system **200** may further include a display unit **214**, such as a liquid crystal display (LCD), an organic light emitting diode (OLED), a flat panel display, a solid state display, a cathode ray tube (CRT), a projector, a printer or other now known or later developed display device for outputting determined information. The display **214** may act as an interface for

the user to see the functioning of the processor **202**, or specifically as an interface with the software stored in the memory **204** or in the drive unit **206**.

[0104] Additionally, the computer system **200** may include an input device **216** configured to allow a user to interact with any of the components of system **200**. The input device **216** may be a number pad, a keyboard, or a cursor control device, such as a mouse, or a joystick, touch screen display, remote control or any other device operative to interact with the system **200**.

[0105] In a particular embodiment, as depicted in FIG. **2**, the computer system **200** may also include a disk or optical drive unit **206**. The disk drive unit **206** may include a computer-readable medium **210** in which one or more sets of instructions **212**, e.g., software, can be embedded. Further, the instructions **212** may embody one or more of the methods or logic as described herein. In a particular embodiment, the instructions **212** may reside completely, or at least partially, within the memory **204** and/or within the processor **202** during execution by the computer system **200**. The memory **204** and the processor **202** also may include computer-readable media as discussed herein.

[0106] The present disclosure contemplates a computer-readable medium that includes instructions **212** or receives and executes instructions **212** responsive to a propagated signal, so that a device connected to a network **220** can communicate voice, video, audio, images or any other data over the network **220**. Further, the instructions **212** may be transmitted or received over the network **220** via a communication interface **218**. The communication interface **218** may be a part of the processor **202** or may be a separate component. The communication interface **218** may be created in software or may be a physical connection in hardware. The communication interface **218** is configured to connect with a network **220**, external media, the display **214**, or any other components in system **200**, or combinations thereof. The connection with the network **220** may be a physical connection, such as a wired Ethernet connection or may be established wirelessly. Likewise, the additional connections with other components of the system **200** may be physical connections or may be established wirelessly.

[0107] The network **220** may include wired networks, wireless networks, or combinations thereof. The wireless network may be a cellular telephone network, an 802.11, 802.16, 802.20, or WiMax network. Further, the network **220** may be a public network, such as the Internet, a private network, such as an intranet, or combinations thereof, and may utilize a variety of networking protocols now available or later developed including, but not limited to, TCP/IP based networking protocols.

[0108] Embodiments of the subject matter and the functional operations described in this specification can be implemented in digital electronic circuitry, or in computer software, firmware, or hardware, including the structures disclosed in this specification and their structural equivalents, or in combinations of one or more of them. Embodiments of the subject matter described in this specification can be implemented as one or more computer program products, i.e., one or more modules of computer program instructions encoded on a computer readable medium for execution by, or to control the operation of, data processing apparatus. While the computer-readable medium is shown to be a single medium, the term “computer-readable medium” includes a single medium or multiple media, such as a centralized or distributed database, and/or associated caches and servers that store one or more sets of instructions. The term “computer-readable medium” shall also include any medium that is capable of storing, encoding or carrying a set of instructions for execution by a processor or that cause a computer system to perform any one or more of the methods or operations disclosed herein. The computer readable medium can be a machine-readable storage device, a machine-readable storage substrate, a memory device, or a combination of one or more of them. The term “data processing apparatus” encompasses all apparatus, devices, and machines for processing data, including by way of example a programmable processor, a computer, or multiple processors or computers. The apparatus can include, in addition to hardware, code that creates an execution environment for the computer program in question, e.g., code that constitutes processor firmware, a protocol stack, a database management system, an operating system, or a

combination of one or more of them.
[0109] In a particular non-limiting, exemplary embodiment, the computer-readable medium can include a solid-state memory such as a memory card or other package that houses one or more non-volatile read-only memories. Further, the computer-readable medium can be a random access memory or other volatile re-writable memory. Additionally, the computer-readable medium can include a magneto-optical or optical medium, such as a disk or tapes or other storage device to capture carrier wave signals such as a signal communicated over a transmission medium. A digital file attachment to an e-mail or other self-contained information archive or set of archives may be considered a distribution medium that is a tangible storage medium. Accordingly, the disclosure is considered to include any one or more of a computer-readable medium or a distribution medium and other equivalents and successor media, in which data or instructions may be stored.
[0110] In an alternative embodiment, dedicated or otherwise specifically configured hardware implementations, such as application specific integrated circuits, programmable logic arrays and other hardware devices, can be constructed to implement one or more of the methods described herein. Applications that may include the apparatus and systems of various embodiments can broadly include a variety of electronic and computer systems. One or more embodiments described herein may implement functions using two or more specific interconnected hardware modules or devices with related control and data signals that can be communicated between and through the modules, or as portions of an application-specific integrated circuit. Accordingly, the present system encompasses software, firmware, and hardware implementations.
[0111] In accordance with various embodiments of the present disclosure, the methods described herein may be implemented by software programs executable by a computer system. Further, in an exemplary, non-limited embodiment, implementations can include distributed processing, component/object distributed processing, and parallel processing. Alternatively, virtual computer system processing can be constructed to implement one or more of the methods or functionality as described herein.
[0112] Although the present specification describes components and functions that may be implemented in particular embodiments with reference to particular standards and protocols, the invention is not limited to such standards and protocols. For example, standards for Internet and other packet switched network transmission (e.g., TCP/IP, UDP/IP, HTML, HTTP, HTTPS) represent examples of the state of the art. Such standards are periodically superseded by faster or more efficient equivalents having essentially the same functions. Accordingly, replacement standards and protocols having the same or similar functions as those disclosed herein are considered equivalents thereof.
[0113] A computer program (also known as a program, software, software application, script, or code) can be written in any form of programming language, including compiled or interpreted languages, and it can be deployed in any form, including as a standalone program or as a module, component, subroutine, or other unit suitable for use in a computing environment. A computer program does not necessarily correspond to a file in a file system. A program can be stored in a portion of a file that holds other programs or data (e.g., one or more scripts stored in a markup language document), in a single file dedicated to the program in question, or in multiple coordinated files (e.g., files that store one or more modules, sub programs, or portions of code). A computer program can be deployed to be executed on one computer or on multiple computers that are located at one site or distributed across multiple sites and interconnected by a communication network.
[0114] The processes and logic flows described in this specification can be performed by one or more programmable processors executing one or more computer programs to perform functions by operating on input data and generating output. The processes and logic flows can also be performed by, and apparatus can also be implemented as, special purpose logic circuitry, e.g., an FPGA (field programmable gate array) or an ASIC (application specific integrated circuit).

[0115] Processors suitable for the execution of a computer program include, by way of example, both general and special purpose microprocessors, and anyone or more processors of any kind of digital computer. Generally, a processor will receive instructions and data from a read only memory or a random access memory or both. The essential elements of a computer are a processor for performing instructions and one or more memory devices for storing instructions and data. Generally, a computer will also include, or be operatively coupled to receive data from or transfer data to, or both, one or more mass storage devices for storing data, e.g., magnetic, magneto optical disks, or optical disks. However, a computer need not have such devices. Moreover, a computer can be embedded in another device, e.g., a mobile telephone, a personal digital assistant (PDA), a mobile audio player, a Global Positioning System (GPS) receiver, to name just a few. Computer readable media suitable for storing computer program instructions and data include all forms of non-volatile memory, media and memory devices, including by way of example semiconductor memory devices, e.g., EPROM, EEPROM, and flash memory devices; magnetic disks, e.g., internal hard disks or removable disks; magneto optical disks; and CD ROM and DVD-ROM disks. The processor and the memory can be supplemented by, or incorporated in, special purpose logic circuitry.

[0116] As used herein, the terms “microprocessor” or “general-purpose processor” (“GPP”) may refer to a hardware device that fetches instructions and data from a memory or storage device and executes those instructions (for example, an Intel Xeon processor or an AMD Opteron processor) to then, for example, process the data in accordance therewith. The term “reconfigurable logic” may refer to any logic technology whose form and function can be significantly altered (i.e., reconfigured) in the field post-manufacture as opposed to a microprocessor, whose function can change post-manufacture, e.g. via computer executable software code, but whose form, e.g. the arrangement/layout and interconnection of logical structures, is fixed at manufacture. The term “software” may refer to data processing functionality that is deployed on a GPP. The term “firmware” may refer to data processing functionality that is deployed on reconfigurable logic. One example of a reconfigurable logic is a field programmable gate array (“FPGA”) which is a reconfigurable integrated circuit. An FPGA may contain programmable logic components called “logic blocks”, and a hierarchy of reconfigurable interconnects that allow the blocks to be “wired together”, somewhat like many (changeable) logic gates that can be inter-wired in (many) different configurations. Logic blocks may be configured to perform complex combinatorial functions, or merely simple logic gates like AND, OR, NOT and XOR. An FPGA may further include memory elements, which may be simple flip-flops or more complete blocks of memory.

[0117] To provide for interaction with a user, embodiments of the subject matter described in this specification can be implemented on a device having a display, e.g., a CRT (cathode ray tube) or LCD (liquid crystal display) monitor, for displaying information to the user and a keyboard and a pointing device, e.g., a mouse or a trackball, by which the user can provide input to the computer. Other kinds of devices can be used to provide for interaction with a user as well. Feedback provided to the user can be any form of sensory feedback, e.g., visual feedback, auditory feedback, or tactile feedback. Input from the user can be received in any form, including acoustic, speech, or tactile input.

[0118] Embodiments of the subject matter described in this specification can be implemented in a computing system that includes a back end component, e.g., a data server, or that includes a middleware component, e.g., an application server, or that includes a front end component, e.g., a client computer having a graphical user interface or a Web browser through which a user can interact with an implementation of the subject matter described in this specification, or any combination of one or more such back end, middleware, or front end components. The components of the system can be interconnected by any form or medium of digital data communication, e.g., a communication network. Examples of communication networks include a local area network (“LAN”) and a wide area network (“WAN”), e.g., the Internet.

[0119] The computing system can include clients and servers. A client and server are generally remote from each other and typically interact through a communication network. The relationship of client and server arises by virtue of computer programs running on the respective computers and having a client-server relationship to each other.
[0120] It should be appreciated that the disclosed embodiments may be applicable to other types of messages depending upon the implementation. Further, the messages may comprise one or more data packets, datagrams or other collection of data formatted, arranged configured and/or packaged in a particular one or more protocols, e.g., the FIX protocol, TCP/IP, Ethernet, etc., suitable for transmission via a network **214** as was described, such as the message format and/or protocols described in U.S. Pat. No. 7,831,491 and U.S. Patent Publication No. 2005/0096999 A1, both of which are incorporated by reference herein in their entireties and relied upon. Further, the disclosed message management system may be implemented using an open message standard implementation, such as FIX, FIX Binary, FIX/FAST, or by an exchange-provided API.
[0121] The embodiments described herein utilize trade related electronic messages such as mass quote messages, individual order messages, modification messages, cancellation messages, etc., so as to enact trading activity in an electronic market. The trading entity and/or market participant may have one or multiple trading terminals associated with the session. Furthermore, the financial instruments may be financial derivative products. Derivative products may include futures contracts, options on futures contracts, futures contracts that are functions of or related to other futures contracts, swaps, swaptions, or other financial instruments that have their price related to or derived from an underlying product, security, commodity, equity, index, or interest rate product. In one embodiment, the orders are for options contracts that belong to a common option class. Orders may also be for baskets, quadrants, other combinations of financial instruments, etc. The option contracts may have a plurality of strike prices and/or comprise put and call contracts. A mass quote message may be received at an exchange. As used herein, an exchange computing system **100** includes a place or system that receives and/or executes orders.
[0122] In an embodiment, a plurality of electronic messages is received from the network. The plurality of electronic messages may be received at a network interface for the electronic trading system. The plurality of electronic messages may be sent from market participants. The plurality of messages may include order characteristics and be associated with actions to be executed with respect to an order that may be extracted from the order characteristics. The action may involve any action as associated with transacting the order in an electronic trading system. The actions may involve placing the orders within a particular market and/or order book of a market in the electronic trading system.
[0123] In an embodiment, the market may operate using characteristics that involve collecting orders over a period of time, such as a batch auction market. In such an embodiment, the period of time may be considered an order accumulation period. The period of time may involve a beginning time and an ending time, with orders placed in the market after the beginning time, and the placed order matched at or after the ending time. As such, the action associated with an order extracted from a message may involve placing the order in the market within the period of time. Also, electronic messages may be received prior to or after the beginning time of the period of time.
[0124] The electronic messages may also include other data relating to the order. In an embodiment, the other data may be data indicating a particular time in which the action is to be executed. As such, the order may be considered a temporally specific order. The particular time in which an action is undertaken may be established with respect to any measure of absolute or relative time. In an embodiment, the time in which an action is undertaken may be established with reference to the beginning time of the time period or ending time of the time period in a batch auction embodiment. For example, the particular time may be a specific amount of time, such as 10 milliseconds, prior to the ending time of an order accumulation period in the batch auction. Further, the order accumulation period may involve dissecting the accumulation period into multiple

consecutive, overlapping, or otherwise divided, sub-periods of time. For example, the sub-periods may involve distinct temporal windows within the order accumulation period. As such, the particular time may be an indicator of a particular temporal window during the accumulation period. For example, the particular time may be specified as the last temporal window prior to the ending time of the accumulation period.

[0125] In an embodiment, the electronic message may also include other actions to be taken with respect to the order. These other actions may be actions to be executed after the initial or primary action associated with the order. For example, the actions may involve modifying or canceling an already placed order. Further, in an embodiment, the other data may indicate order modification characteristics. For example, the other data may include a price or volume change in an order. The other actions may involve modifying the already placed order to align with the order modification characteristics, such as changing the price or volume of the already placed order.

[0126] In an embodiment, other actions may be dependent actions. For example, the execution of the actions may involve a detection of an occurrence of an event. Such triggering events may be described as other data in the electronic message. For example, the triggering event may be a release of an economic statistic from an organization relating to a product being bought or sold in the electronic market, a receipt of pricing information from a correlated electronic market, a detection of a change in market sentiment derived from identification of keywords in social media or public statements of officials related to a product being bought or sold in the electronic market, and/or any other event or combination of events which may be detected by an electronic trading system.

[0127] In an embodiment, the action, or a primary action, associated with an order may be executed. For example, an order extracted from electronic message order characteristics may be placed into a market, or an electronic order book for a market, such that the order may be matched with other orders counter thereto.

[0128] In an embodiment involving a market operating using batch auction principles, the action, such as placing the order, may be executed subsequent to the beginning time of the order accumulation period, but prior to the ending time of the order accumulation period. Further, as indicated above, a message may also include other information for the order, such as a particular time the action is to be executed. In such an embodiment, the action may be executed at the particular time. For example, in an embodiment involving a batch auction process having sub-periods during an order accumulation period, an order may be placed during a specified sub-period of the order accumulation period. The disclosed embodiments may be applicable to batch auction processing, as well as continuous processing.

[0129] Also, it may be noted that messages may be received prior or subsequent to the beginning time of an order accumulation period. Orders extracted from messages received prior to the beginning time may have the associated actions, or primary actions such as placing the order, executed at any time subsequent to the beginning time, but prior to the ending time, of the order accumulation period when no particular time for the execution is indicated in the electronic message. In an embodiment, messages received prior to the beginning time but not having a particular time specified will have the associated action executed as soon as possible after the beginning time. Because of this, specifying a time for order action execution may allow a distribution and more definite relative time of order placement so as to allow resources of the electronic trading system to operate more efficiently.

[0130] In an embodiment, the execution of temporally specific messages may be controlled by the electronic trading system such that a limited or maximum number may be executed in any particular accumulation period, or sub-period. In an embodiment, the order accumulation time period involves a plurality of sub-periods involving distinct temporal windows, a particular time indicated by a message may be indicative of a particular temporal window of the plurality of temporal windows, and the execution of the at least one temporally specific message is limited to

the execution of a specified sub-period maximum number of temporally specific messages during a particular sub-period. The electronic trading system may distribute the ability to submit temporally specific message to selected market participants. For example, only five temporally specific messages may be allowed in any one particular period or sub-period. Further, the ability to submit temporally specific messages within particular periods or sub-periods may be distributed based on any technique. For example, the temporally specific messages for a particular sub-period may be auctioned off or otherwise sold by the electronic trading system to market participants. Also, the electronic trading system may distribute the temporally specific messages to preferred market participants, or as an incentive to participate in a particular market.

[0131] In an embodiment, an event occurrence may be detected. The event occurrence may be the occurrence of an event that was specified as other information relating to an order extracted from an electronic message. The event may be a triggering event for a modification or cancellation action associated with an order. The event may be detected subsequent to the execution of the first action when an electronic message further comprises the data representative of the event and a secondary action associated with the order. In an embodiment involving a market operating on batch auction principles, the event may be detected subsequent to the execution of a first action, placing an order, but prior to the ending time of an order accumulation period in which the action was executed.

[0132] In an embodiment, other actions associated with an order may be executed. The other actions may be any action associated with an order. For example, the action may be a conditional action that is executed in response to a detection of an occurrence of an event. Further, in a market operating using batch auction principles, the conditional action may be executed after the placement of an order during an order accumulation period, but in response to a detection of an occurrence of an event prior to an ending time of the order accumulation period. In such an embodiment, the conditional action may be executed prior to the ending time of the order accumulation period. For example, the placed order may be canceled, or modified using other provided order characteristics in the message, in response to the detection of the occurrence of the event.

[0133] An event may be a release of an economic statistic or a fluctuation of prices in a correlated market. An event may also be a perceptible change in market sentiment of a correlated market. A change may be perceptible based on a monitoring of orders or social media for keywords in reference to the market in question. For example, electronic trading systems may be configured to be triggered for action by a use of keywords during a course of ongoing public statements of officials who may be in a position to impact markets, such as Congressional testimony of the Chairperson of the Federal Reserve System.

[0134] The other or secondary action may also be considered a modification of a first action executed with respect to an order. For example, a cancellation may be considered a cancellation of the placement of the order. Further, a secondary action may have other data in the message which indicates a specific time in which the secondary action may be executed. The specific time may be a time relative to a first action, or placement of the order, or relative to an accumulation period in a batch auction market. For example, the specific time for execution of the secondary action may be at a time specified relative and prior to the ending period of the order accumulation period. Further, multiple secondary actions may be provided for a single order. Also, with each secondary action a different triggering event may be provided.

[0135] In an embodiment, an incoming transaction may be received. The incoming transaction may be from, and therefore associated with, a market participant of an electronic market managed by an electronic trading system. The transaction may involve an order as extracted from a received message, and may have an associated action. The actions may involve placing an order to buy or sell a financial product in the electronic market, or modifying or deleting such an order. In an embodiment, the financial product may be based on an associated financial instrument which the

electronic market is established to trade.
[0136] In an embodiment, the action associated with the transaction is determined. For example, it may be determined whether the incoming transaction comprises an order to buy or sell a quantity of the associated financial instrument or an order to modify or cancel an existing order in the electronic market. Orders to buy or sell and orders to modify or cancel may be acted upon differently by the electronic market. For example, data indicative of different characteristics of the types of orders may be stored.
[0137] In an embodiment, data relating to the received transaction is stored. The data may be stored in any device, or using any technique, operable to store and provide recovery of data. For example, a memory **204** or computer readable medium **210**, may be used to store data, as is described with respect to FIG. **2** in further detail herein. Data may be stored relating received transactions for a period of time, indefinitely, or for a rolling most recent time period such that the stored data is indicative of the market participant's recent activity in the electronic market.
[0138] If and/or when a transaction is determined to be an order to modify or cancel a previously placed, or existing, order, data indicative of these actions may be stored. For example, data indicative of a running count of a number or frequency of the receipt of modify or cancel orders from the market participant may be stored. A number may be a total number of modify or cancel orders received from the market participant, or a number of modify or cancel orders received from the market participant over a specified time. A frequency may be a time based frequency, as in a number of cancel or modify orders per unit of time, or a number of cancel or modify orders received from the market participant as a percentage of total transactions received from the participant, which may or may not be limited by a specified length of time.
[0139] If and/or when a transaction is determined to be an order to buy or sell a financial product, or financial instrument, other indicative data may be stored. For example, data indicative of quantity and associated price of the order to buy or sell may be stored.
[0140] Data indicative of attempts to match incoming orders may also be stored. The data may be stored in any device, or using any technique, operable to store and provide recovery of data. For example, a memory **204** or computer readable medium **210**, may be used to store data, as is described with respect to FIG. **2**. The acts of the process as described herein may also be repeated. As such, data for multiple received transactions for multiple market participants may be stored and used as describe herein.
[0141] The order processing module **118** may also store data indicative of characteristics of the extracted orders. For example, the order processing module may store data indicative of orders having an associated modify or cancel action, such as by recording a count of the number of such orders associated with particular market participants. The order processing module may also store data indicative of quantities and associated prices of orders to buy or sell a product placed in the market order book **110**, as associated with particular market participants.
[0142] Also, the order processing module **118** may be configured to calculate and associate with particular orders a value indicative of an associated market participant's market activity quality, which is a value indicative of whether the market participant's market activity increases or tends to increase liquidity of a market. This value may be determined based on the price of the particular order, previously stored quantities of orders from the associated market participant, the previously stored data indicative of previously received orders to modify or cancel as associated with the market participant, and previously stored data indicative of a result of the attempt to match previously received orders stored in association with the market participant. The order processing module **118** may determine or otherwise calculate scores indicative of the quality value based on these stored extracted order characteristics, such as an MQI as described herein.
[0143] Further, electronic trading systems may perform actions on orders placed from received messages based on various characteristics of the messages and/or market participants associated with the messages. These actions may include matching the orders either during a continuous

auction process, or at the conclusion of a collection period during a batch auction process. The matching of orders may be by any technique.

[0144] The matching of orders may occur based on a priority indicated by the characteristics of orders and market participants associated with the orders. Orders having a higher priority may be matched before orders of a lower priority. Such priority may be determined using various techniques. For example, orders that were indicated by messages received earlier may receive a higher priority to match than orders that were indicated by messages received later. Also, scoring or grading of the characteristics may provide for priority determination. Data indicative of order matches may be stored by a match engine and/or an order processing module **118**, and used for determining MQI scores of market participants.

Example Users

[0145] Generally, a market may involve market makers, such as market participants who consistently provide bids and/or offers at specific prices in a manner typically conducive to balancing risk, and market takers who may be willing to execute transactions at prevailing bids or offers may be characterized by more aggressive actions so as to maintain risk and/or exposure as a speculative investment strategy. From an alternate perspective, a market maker may be considered a market participant who places an order to sell at a price at which there is no previously or concurrently provided counter order. Similarly, a market taker may be considered a market participant who places an order to buy at a price at which there is a previously or concurrently provided counter order. A balanced and efficient market may involve both market makers and market takers, coexisting in a mutually beneficial basis. The mutual existence, when functioning properly, may facilitate liquidity in the market such that a market may exist with “tight” bid-ask spreads (e.g., small difference between bid and ask prices) and a “deep” volume from many currently provided orders such that large quantity orders may be executed without driving prices significantly higher or lower.

[0146] As such, both market participant types are useful in generating liquidity in a market, but specific characteristics of market activity taken by market participants may provide an indication of a particular market participant's effect on market liquidity. For example, a Market Quality Index (“MQI”) of an order may be determined using the characteristics. An MQI may be considered a value indicating a likelihood that a particular order will improve or facilitate liquidity in a market. That is, the value may indicate a likelihood that the order will increase a probability that subsequent requests and transaction from other market participants will be satisfied. As such, an MQI may be determined based on a proximity of the entered price of an order to a midpoint of a current bid-ask price spread, a size of the entered order, a volume or quantity of previously filled orders of the market participant associated with the order, and/or a frequency of modifications to previous orders of the market participant associated with the order. In this way, an electronic trading system may function to assess and/or assign an MQI to received electronic messages to establish messages that have a higher value to the system, and thus the system may use computing resources more efficiently by expending resources to match orders of the higher value messages prior to expending resources of lower value messages.

[0147] While an MQI may be applied to any or all market participants, such an index may also be applied only to a subset thereof, such as large market participants, or market participants whose market activity as measured in terms of average daily message traffic over a limited historical time period exceeds a specified number. For example, a market participant generating more than 500, 1,000, or even 10,000 market messages per day may be considered a large market participant.

[0148] An exchange provides one or more markets for the purchase and sale of various types of products including financial instruments such as stocks, bonds, futures contracts, options, currency, cash, and other similar instruments. Agricultural products and commodities are also examples of products traded on such exchanges. A futures contract is a product that is a contract for the future delivery of another financial instrument such as a quantity of grains, metals, oils, bonds, currency,

or cash. Generally, each exchange establishes a specification for each market provided thereby that defines at least the product traded in the market, minimum quantities that must be traded, and minimum changes in price (e.g., tick size). For some types of products (e.g., futures or options), the specification further defines a quantity of the underlying product represented by one unit (or lot) of the product, and delivery and expiration dates. As will be described, the exchange may further define the matching algorithm, or rules, by which incoming orders will be matched/allocated to resting orders.

Matching and Transaction Processing

[0149] Market participants, e.g., traders, use software to send orders or messages to the trading platform. The order identifies the product, the quantity of the product the trader wishes to trade, a price at which the trader wishes to trade the product, and a direction of the order (i.e., whether the order is a bid, i.e., an offer to buy, or an ask, i.e., an offer to sell). It will be appreciated that there may be other order types or messages that traders can send including requests to modify or cancel a previously submitted order.

[0150] The exchange computer system monitors incoming orders received thereby and attempts to identify, i.e., match or allocate, as described herein, one or more previously received, but not yet matched, orders, i.e., limit orders to buy or sell a given quantity at a given price, referred to as "resting" orders, stored in an order book database, wherein each identified order is contra to the incoming order and has a favorable price relative to the incoming order. An incoming order may be an "aggressor" order, i.e., a market order to sell a given quantity at whatever may be the current resting bid order price(s) or a market order to buy a given quantity at whatever may be the current resting ask order price(s). An incoming order may be a "market making" order, i.e., a market order to buy or sell at a price for which there are currently no resting orders. In particular, if the incoming order is a bid, i.e., an offer to buy, then the identified order(s) will be an ask, i.e., an offer to sell, at a price that is identical to or higher than the bid price. Similarly, if the incoming order is an ask, i.e., an offer to sell, the identified order(s) will be a bid, i.e., an offer to buy, at a price that is identical to or lower than the offer price.

[0151] An exchange computing system may receive conditional orders or messages for a data object, where the order may include two prices or values: a reference value and a stop value. A conditional order may be configured so that when a product represented by the data object trades at the reference price, the stop order is activated at the stop value. For example, if the exchange computing system's order management module includes a stop order with a stop price of 5 and a limit price of 1 for a product, and a trade at 5 (i.e., the stop price of the stop order) occurs, then the exchange computing system attempts to trade at 1 (i.e., the limit price of the stop order). In other words, a stop order is a conditional order to trade (or execute) at the limit price that is triggered (or elected) when a trade at the stop price occurs.

[0152] Stop orders also rest on, or are maintained in, an order book to monitor for a trade at the stop price, which triggers an attempted trade at the limit price. In some embodiments, a triggered limit price for a stop order may be treated as an incoming order.

[0153] Upon identification (matching) of a contra order(s), a minimum of the quantities associated with the identified order and the incoming order is matched and that quantity of each of the identified and incoming orders become two halves of a matched trade that is sent to a clearing house. The exchange computer system considers each identified order in this manner until either all of the identified orders have been considered or all of the quantity associated with the incoming order has been matched, i.e., the order has been filled. If any quantity of the incoming order remains, an entry may be created in the order book database and information regarding the incoming order is recorded therein, i.e., a resting order is placed on the order book for the remaining quantity to await a subsequent incoming order counter thereto.

[0154] It should be appreciated that in electronic trading systems implemented via an exchange computing system, a trade price (or match value) may differ from (i.e., be better for the submitter,

e.g., lower than a submitted buy price or higher than a submitted sell price) the limit price that is submitted, e.g., a price included in an incoming message, or a triggered limit price from a stop order.

[0155] As used herein, “better” than a reference value means lower than the reference value if the transaction is a purchase (or acquire) transaction, and higher than the reference value if the transaction is a sell transaction. Said another way, for purchase (or acquire) transactions, lower values are better, and for relinquish or sell transactions, higher values are better.

[0156] Traders access the markets on a trading platform using trading software that receives and displays at least a portion of the order book for a market, i.e., at least a portion of the currently resting orders, enables a trader to provide parameters for an order for the product traded in the market, and transmits the order to the exchange computer system. The trading software typically includes a graphical user interface to display at least a price and quantity of some of the entries in the order book associated with the market. The number of entries of the order book displayed is generally preconfigured by the trading software, limited by the exchange computer system, or customized by the user. Some graphical user interfaces display order books of multiple markets of one or more trading platforms. The trader may be an individual who trades on his/her behalf, a broker trading on behalf of another person or entity, a group, or an entity. Furthermore, the trader may be a system that automatically generates and submits orders.

[0157] If the exchange computer system identifies that an incoming market order may be filled by a combination of multiple resting orders, e.g., the resting order at the best price only partially fills the incoming order, the exchange computer system may allocate the remaining quantity of the incoming, i.e., that which was not filled by the resting order at the best price, among such identified orders in accordance with prioritization and allocation rules/algorithms, referred to as “allocation algorithms” or “matching algorithms,” as, for example, may be defined in the specification of the particular financial product or defined by the exchange for multiple financial products. Similarly, if the exchange computer system identifies multiple orders contra to the incoming limit order and that have an identical price which is favorable to the price of the incoming order, i.e., the price is equal to or better, e.g., lower if the incoming order is a buy (or instruction to purchase, or instruction to acquire) or higher if the incoming order is a sell (or instruction to relinquish), than the price of the incoming order, the exchange computer system may allocate the quantity of the incoming order among such identified orders in accordance with the matching algorithms as, for example, may be defined in the specification of the particular financial product or defined by the exchange for multiple financial products.

[0158] An exchange responds to inputs, such as trader orders, cancellation, etc., in a manner as expected by the market participants, such as based on market data, e.g., prices, available counter-orders, etc., to provide an expected level of certainty that transactions will occur in a consistent and predictable manner and without unknown or unascertainable risks. Accordingly, the method by which incoming orders are matched with resting orders must be defined so that market participants have an expectation of what the result will be when they place an order or have resting orders and an incoming order is received, even if the expected result is, in fact, at least partially unpredictable due to some component of the process being random or arbitrary or due to market participants having imperfect or less than all information, e.g., unknown position of an order in an order book. Typically, the exchange defines the matching/allocation algorithm that will be used for a particular financial product, with or without input from the market participants. Once defined for a particular product, the matching/allocation algorithm is typically not altered, except in limited circumstance, such as to correct errors or improve operation, so as not to disrupt trader expectations. It will be appreciated that different products offered by a particular exchange may use different matching algorithms.

[0159] For example, a first-in/first-out (FIFO) matching algorithm, also referred to as a “Price Time” algorithm, considers each identified order sequentially in accordance with when the

identified order was received. The quantity of the incoming order is matched to the quantity of the identified order at the best price received earliest, then quantities of the next earliest best price orders, and so on until the quantity of the incoming order is exhausted. Some product specifications define the use of a pro-rata matching algorithm, wherein a quantity of an incoming order is allocated to each of plurality of identified orders proportionally. Some exchange computer systems provide a priority to certain standing orders in particular markets. An example of such an order is the first order that improves a price (i.e., improves the market) for the product during a trading session. To be given priority, the trading platform may require that the quantity associated with the order is at least a minimum quantity. Further, some exchange computer systems cap the quantity of an incoming order that is allocated to a standing order on the basis of a priority for certain markets. In addition, some exchange computer systems may give a preference to orders submitted by a trader who is designated as a market maker for the product. Other exchange computer systems may use other criteria to determine whether orders submitted by a particular trader are given a preference. Typically, when the exchange computer system allocates a quantity of an incoming order to a plurality of identified orders at the same price, the trading host allocates a quantity of the incoming order to any orders that have been given priority. The exchange computer system thereafter allocates any remaining quantity of the incoming order to orders submitted by traders designated to have a preference, and then allocates any still remaining quantity of the incoming order using the FIFO or pro-rata algorithms. Pro-rata algorithms used in some markets may require that an allocation provided to a particular order in accordance with the pro-rata algorithm must meet at least a minimum allocation quantity. Any orders that do not meet or exceed the minimum allocation quantity are allocated to on a FIFO basis after the pro-rata allocation (if any quantity of the incoming order remains). More information regarding order allocation may be found in U.S. Pat. No. 7,853,499, the entirety of which is incorporated by reference herein and relied upon.

[0160] Other examples of matching algorithms which may be defined for allocation of orders of a particular financial product include: [0161] Price Explicit Time [0162] Order Level Pro Rata [0163] Order Level Priority Pro Rata [0164] Preference Price Explicit Time [0165] Preference Order Level Pro Rata [0166] Preference Order Level Priority Pro Rata [0167] Threshold Pro-Rata [0168] Priority Threshold Pro-Rata [0169] Preference Threshold Pro-Rata [0170] Priority Preference Threshold Pro-Rata [0171] Split Price-Time Pro-Rata

[0172] For example, the Price Explicit Time trading policy is based on the basic Price Time trading policy with Explicit Orders having priority over Implied Orders at the same price level. The order of traded volume allocation at a single price level may therefore be:

Explicit Order with Oldest Timestamp First. Followed by

[0173] Any remaining explicit orders in timestamp sequence (First In, First Out—FIFO) next. Followed by

Implied Order with Oldest Timestamp Next. Followed by

[0174] Any remaining implied orders in timestamp sequence (FIFO).

[0175] In Order Level Pro Rata, also referred to as Price Pro Rata, priority is given to orders at the best price (highest for a bid, lowest for an offer). If there are several orders at this best price, equal priority is given to every order at this price and incoming business is divided among these orders in proportion to their order size. The Pro Rata sequence of events is:

[0176] 1. Extract all potential matching orders at best price from the order book into a list.

[0177] 2. Sort the list by order size, largest order size first. If equal order sizes, oldest timestamp first. This is the matching list.

[0178] 3. Find the ‘Matching order size, which is the total size of all the orders in the matching list.

[0179] 4. Find the ‘tradable volume’, which is the smallest of the matching volume and the volume left to trade on the incoming order.

[0180] 5. Allocate volume to each order in the matching list in turn, starting at the beginning of the list. If all the tradable volume gets used up, orders near the end of the list may not get allocation.

[0181] 6. The amount of volume to allocate to each order is given by the formula:

(Order volume/Matching volume)*Tradable volume [0182] The result is rounded down (for example, 21.99999999 becomes 21) unless the result is less than 1, when it becomes 1.
[0183] 7. If tradable volume remains when the last order in the list had been allocated to, return to step 3.
[0184] Note: The matching list is not re-sorted, even though the volume has changed. The order which originally had the largest volume is still at the beginning of the list.
[0185] 8. If there is still volume left to trade on the incoming order, repeat the entire algorithm at the next price level.
[0186] Order Level Priority Pro Rata, also referred to as Threshold Pro Rata, is similar to the Price (or 'Vanilla') Pro Rata algorithm but has a volume threshold defined. Any pro rata allocation below the threshold will be rounded down to 0. The initial pass of volume allocation is carried out in using pro rata; the second pass of volume allocation is carried out using Price Explicit Time. The Threshold Pro Rata sequence of events is:
[0187] 1. Extract all potential matching orders at best price from the order book into a list.
[0188] 2. Sort the list by explicit time priority, oldest timestamp first. This is the matching list.
[0189] 3. Find the 'Matching volume', which is the total volume of all the orders in the matching list.
[0190] 4. Find the 'tradable volume', which is the smallest of the matching volume and the volume left to trade on the incoming order.
[0191] 5. Allocate volume to each order in the matching list in turn, starting at the beginning of the list.
[0192] 6. The amount of volume to allocate to each order is given by the formula:

(Order volume/Matching volume)*Tradable volume
[0193] The result is rounded down to the nearest lot (for example, 21.99999999 becomes 21) unless the result is less than the defined threshold in which case it is rounded down to 0.
[0194] 7. If tradable volume remains when the last order in the list had been allocated to, the remaining volume is allocated in time priority to the matching list.
[0195] 8. If there is still volume left to trade on the incoming order, repeat the entire algorithm at the next price level.
[0196] In the Split Price Time Pro-Rata algorithms, a Price Time Percentage parameter is defined. This percentage of the matching volume at each price is allocated by the Price Explicit Time algorithm and the remainder is allocated by the Threshold Pro-Rata algorithm. There are four variants of this algorithm, with and without Priority and/or Preference. The Price Time Percentage parameter is an integer between 1 and 99. (A percentage of zero would be equivalent to using the respective existing Threshold Pro-Rata algorithm, and a percentage of 100 would be equivalent to using the respective existing Price Time algorithm). The Price Time Volume will be the residual incoming volume, after any priority and/or Preference allocation has been made, multiplied by the Price Time Percentage. Fractional parts will be rounded up, so the Price Time Volume will always be at least 1 lot and may be the entire incoming volume. The Price Time Volume is allocated to resting orders in strict time priority. Any remaining incoming volume after the Price Time Volume has been allocated will be allocated according to the respective Threshold Pro-Rata algorithm. The sequence of allocation, at each price level, is therefore: [0197] 1. Priority order, if applicable [0198] 2. Preference allocation, if applicable [0199] 3. Price Time allocation of the configured percentage of incoming volume [0200] 4. Threshold Pro-Rata allocation of any remaining incoming volume
[0201] 5. Final allocation of any leftover lots in time sequence.
[0202] Any resting order may receive multiple allocations from the various stages of the algorithm.
[0203] It will be appreciated that there may be other allocation algorithms, including combinations

of algorithms, now available or later developed, which may be utilized with the disclosed embodiments, and all such algorithms are contemplated herein. In one embodiment, the disclosed embodiments may be used in any combination or sequence with the allocation algorithms described herein.

[0204] One exemplary system for matching is described in U.S. patent application Ser. No. 13/534,499, filed on Jun. 27, 2012, entitled “Multiple Trade Matching Algorithms,” published as U.S. Patent Application Publication No. 2014/0006243 A1, the entirety of which is incorporated by reference herein and relied upon, discloses an adaptive match engine which draws upon different matching algorithms, e.g., the rules which dictate how a given order should be allocated among qualifying resting orders, depending upon market conditions, to improve the operation of the market. For example, for a financial product, such as a futures contract, having a future expiration date, the match engine may match incoming orders according to one algorithm when the remaining time to expiration is above a threshold, recognizing that during this portion of the life of the contract, the market for this product is likely to have high volatility. However, as the remaining time to expiration decreases, volatility may decrease. Accordingly, when the remaining time to expiration falls below the threshold, the match engine switches to a different match algorithm which may be designed to encourage trading relative to the declining trading volatility. Thereby, by conditionally switching among matching algorithms within the same financial product, as will be described, the disclosed match engine may automatically adapt to the changing market conditions of a financial product, e.g., a limited life product, in a non-preferential manner, maintaining fair order allocation while improving market liquidity, e.g., over the life of the product.

[0205] In one implementation, the system may evaluate market conditions on a daily basis and, based thereon, change the matching algorithm between daily trading sessions, i.e., when the market is closed, such that when the market reopens, a new trading algorithm is in effect for the particular product. As will be described, the disclosed embodiments may facilitate more frequent changes to the matching algorithms so as to dynamically adapt to changing market conditions, e.g., intra-day changes, and even intra-order matching changes. It will be further appreciated that hybrid matching algorithms, which match part of an order using one algorithm and another part of the order using a different algorithm, may also be used.

[0206] With respect to incoming orders, some traders, such as automated and/or algorithmic traders, attempt to respond to market events, such as to capitalize upon a mispriced resting order or other market inefficiency, as quickly as possible. This may result in penalizing the trader who makes an errant trade, or whose underlying trading motivations have changed, and who cannot otherwise modify or cancel their order faster than other traders can submit trades there against. It may considered that an electronic trading system that rewards the trader who submits their order first creates an incentive to either invest substantial capital in faster trading systems, participate in the market substantially to capitalize on opportunities (aggressor side/lower risk trading) as opposed to creating new opportunities (market making/higher risk trading), modify existing systems to streamline business logic at the cost of trade quality, or reduce one's activities and exposure in the market. The result may be a lesser quality market and/or reduced transaction volume, and corresponding thereto, reduced fees to the exchange.

[0207] With respect to resting orders, allocation/matching suitable resting orders to match against an incoming order can be performed, as described herein, in many different ways. Generally, it will be appreciated that allocation/matching algorithms are only needed when the incoming order quantity is less than the total quantity of the suitable resting orders as, only in this situation, is it necessary to decide which resting order(s) will not be fully satisfied, which trader(s) will not get their orders filled. It can be seen from the above descriptions of the matching/allocation algorithms, that they fall generally into three categories: time priority/first-in-first-out (“FIFO”), pro rata, or a hybrid of FIFO and pro rata.

[0208] As described above, matching systems apply a single algorithm, or combined algorithm, to

all of the orders received for a particular financial product to dictate how the entire quantity of the incoming order is to be matched/allocated. In contrast, the disclosed embodiments may apply different matching algorithms, singular or combined, to different orders, as will be described, recognizing that the allocation algorithms used by the trading host for a particular market may, for example, affect the liquidity of the market. Specifically, some allocation algorithms may encourage traders to submit more orders, where each order is relatively small, while other allocation algorithms encourage traders to submit larger orders. Other allocation algorithms may encourage a trader to use an electronic trading system that can monitor market activity and submit orders on behalf of the trader very quickly and without intervention. As markets and technologies available to traders evolve, the allocation algorithms used by trading hosts must also evolve accordingly to enhance liquidity and price discovery in markets, while maintaining a fair and equitable market.

[0209] FIFO generally rewards the first trader to place an order at a particular price and maintains this reward indefinitely. So if a trader is the first to place an order at price X, no matter how long that order rests and no matter how many orders may follow at the same price, as soon as a suitable incoming order is received, that first trader will be matched first. This “first mover” system may commit other traders to positions in the queue after the first move traders. Furthermore, while it may be beneficial to give priority to a trader who is first to place an order at a given price because that trader is, in effect, taking a risk, the longer that the trader's order rests, the less beneficial it may be. For instance, it could deter other traders from adding liquidity to the marketplace at that price because they know the first mover (and potentially others) already occupies the front of the queue.

[0210] With a pro rata allocation, incoming orders are effectively split among suitable resting orders. This provides a sense of fairness in that everyone may get some of their order filled. However, a trader who took a risk by being first to place an order (a “market turning” order) at a price may end up having to share an incoming order with a much later submitted order. Furthermore, as a pro rata allocation distributes the incoming order according to a proportion based on the resting order quantities, traders may place orders for large quantities, which they are willing to trade but may not necessarily want to trade, in order to increase the proportion of an incoming order that they will receive. This results in an escalation of quantities on the order book and exposes a trader to a risk that someone may trade against one of these orders and subject the trader to a larger trade than they intended. In the typical case, once an incoming order is allocated against these large resting orders, the traders subsequently cancel the remaining resting quantity which may frustrate other traders. Accordingly, as FIFO and pro rata both have benefits and problems, exchanges may try to use hybrid allocation/matching algorithms which attempt to balance these benefits and problems by combining FIFO and pro rata in some manner. However, hybrid systems define conditions or fixed rules to determine when FIFO should be used and when pro rata should be used. For example, a fixed percentage of an incoming order may be allocated using a FIFO mechanism with the remainder being allocated pro rata.

Spread Instruments

[0211] Traders trading on an exchange including, for example, exchange computer system **100**, often desire to trade multiple financial instruments in combination. Each component of the combination may be called a leg. Traders can submit orders for individual legs or in some cases can submit a single order for multiple financial instruments in an exchange-defined combination. Such orders may be called a strategy order, a spread order, or a variety of other names.

[0212] A spread instrument may involve the simultaneous purchase of one security and sale of a related security, called legs, as a unit. The legs of a spread instrument may be options or futures contracts, or combinations of the two. Trades in spread instruments are executed to yield an overall net position whose value, called the spread, depends on the difference between the prices of the legs. Spread instruments may be traded in an attempt to profit from the widening or narrowing of the spread, rather than from movement in the prices of the legs directly. Spread instruments are

either “bought” or “sold” depending on whether the trade will profit from the widening or narrowing of the spread, respectively. An exchange often supports trading of common spreads as a unit rather than as individual legs, thus ensuring simultaneous execution of the two legs, eliminating the execution risk of one leg executing but the other failing.

[0213] One example of a spread instrument is a calendar spread instrument. The legs of a calendar spread instrument differ in delivery date of the underlier. The leg with the earlier occurring delivery date is often referred to as the lead month contract. A leg with a later occurring delivery date is often referred to as a deferred month contract. Another example of a spread instrument is a butterfly spread instrument, which includes three legs having different delivery dates. The delivery dates of the legs may be equidistant to each other. The counterparty orders that are matched against such a combination order may be individual, “outright” orders or may be part of other combination orders.

[0214] In other words, an exchange may receive, and hold or let rest on the books, outright orders for individual contracts as well as outright orders for spreads associated with the individual contracts. An outright order (for either a contract or for a spread) may include an outright bid or an outright offer, although some outright orders may bundle many bids or offers into one message (often called a mass quote).

[0215] A spread is an order for the price difference between two contracts. This results in the trader holding a long and a short position in two or more related futures or options on futures contracts, with the objective of profiting from a change in the price relationship. A typical spread product includes multiple legs, each of which may include one or more underlying financial instruments. A butterfly spread product, for example, may include three legs. The first leg may consist of buying a first contract. The second leg may consist of selling two of a second contract. The third leg may consist of buying a third contract. The price of a butterfly spread product may be calculated as:

$$\text{Butterfly}=\text{Leg1}-2\times\text{Leg2}+\text{Leg3} \qquad \text{(equation 1)}$$

[0216] In the above equation, Leg1 equals the price of the first contract, Leg2 equals the price of the second contract and Leg3 equals the price of the third contract. Thus, a butterfly spread could be assembled from two inter-delivery spreads in opposite directions with the center delivery month common to both spreads.

[0217] A calendar spread, also called an intra-commodity spread, for futures is an order for the simultaneous purchase and sale of the same futures contract in different contract months (i.e., buying a September CME S&P 500® futures contract and selling a December CME S&P 500 futures contract).

[0218] A crush spread is an order, usually in the soybean futures market, for the simultaneous purchase of soybean futures and the sale of soybean meal and soybean oil futures to establish a processing margin. A crack spread is an order for a specific spread trade involving simultaneously buying and selling contracts in crude oil and one or more derivative products, typically gasoline and heating oil. Oil refineries may trade a crack spread to hedge the price risk of their operations, while speculators attempt to profit from a change in the oil/gasoline price differential.

[0219] A straddle is an order for the purchase or sale of an equal number of puts and calls, with the same strike price and expiration dates. A long straddle is a straddle in which a long position is taken in both a put and a call option. A short straddle is a straddle in which a short position is taken in both a put and a call option. A strangle is an order for the purchase of a put and a call, in which the options have the same expiration and the put strike is lower than the call strike, called a long strangle. A strangle may also be the sale of a put and a call, in which the options have the same expiration and the put strike is lower than the call strike, called a short strangle. A pack is an order for the simultaneous purchase or sale of an equally weighted, consecutive series of four futures contracts, quoted on an average net change basis from the previous day's settlement price. Packs provide a readily available, widely accepted method for executing multiple futures contracts with a

single transaction. A bundle is an order for the simultaneous sale or purchase of one each of a series of consecutive futures contracts. Bundles provide a readily available, widely accepted method for executing multiple futures contracts with a single transaction.

Implication

[0220] Thus an exchange may match outright orders, such as individual contracts or spread orders (which as discussed herein could include multiple individual contracts). The exchange may also imply orders from outright orders. For example, exchange computer system **100** may derive, identify and/or advertise, publish, display or otherwise make available for trading orders based on outright orders.

[0221] As was described above, the financial instruments which are the subject of the orders to trade, may include one or more component financial instruments. While each financial instrument may have its own order book, i.e. market, in which it may be traded, in the case of a financial instrument having more than one component financial instrument, those component financial instruments may further have their own order books in which they may be traded. Accordingly, when an order for a financial instrument is received, it may be matched against a suitable counter order in its own order book or, possibly, against a combination of suitable counter orders in the order books the component financial instruments thereof, or which share a common component financial instrument. For example, an order for a spread contract comprising component financial instruments A and B may be matched against another suitable order for that spread contract. However, it may also be matched against suitable separate counter orders for the A and for the B component financial instruments found in the order books therefore. Similarly, if an order for the A contract is received and suitable match cannot be found in the A order book, it may be possible to match order for A against a combination of a suitable counter order for a spread contract comprising the A and B component financial instruments and a suitable counter order for the B component financial instrument. This is referred to as “implication” where a given order for a financial instrument may be matched via a combination of suitable counter orders for financial instruments which share common, or otherwise interdependent, component financial instruments. Implication increases the liquidity of the market by providing additional opportunities for orders to be traded. Increasing the number of transactions may further increase the number of transaction fees collected by the electronic trading system.

[0222] The order for a particular financial instrument actually received from a market participant, whether it comprises one or more component financial instruments, is referred to as a “real” or “outright” order, or simply as an outright. The one or more orders which must be synthesized and submitted into order books other than the order book for the outright order in order to create matches therein, are referred to as “implied” orders. Upon receipt of an incoming order, the identification or derivation of suitable implied orders which would allow at least a partial trade of the incoming outright order to be executed is referred to as “implication” or “implied matching”, the identified orders being referred to as an “implied match.” Depending on the number component financial instruments involved, and whether those component financial instruments further comprise component financial instruments of their own, there may be numerous different implied matches identified which would allow the incoming order to be at least partially matched and mechanisms may be provided to arbitrate, e.g., automatically, among them, such as by picking the implied match comprising the least number of component financial instruments or the least number of synthesized orders.

[0223] Upon receipt of an incoming order, or thereafter, a combination of one or more suitable/hypothetical counter orders which have not actually been received but if they were received, would allow at least a partial trade of the incoming order to be executed, may be, e.g., automatically, identified or derived and referred to as an “implied opportunity.” As with implied matches, there may be numerous implied opportunities identified for a given incoming order. Implied opportunities are advertised to the market participants, such as via suitable synthetic

orders, e.g. counter to the desired order, being placed on the respective order books to rest (or give the appearance that there is an order resting) and presented via the market data feed, electronically communicated to the market participants, to appear available to trade in order to solicit the desired orders from the market participants. Depending on the number component financial instruments involved, and whether those component financial instruments further comprise component financial instruments of their own, there may be numerous implied opportunities, the submission of a counter order in response thereto, would allow the incoming order to be at least partially matched.
[0224] Implied opportunities, e.g. the advertised synthetic orders, may frequently have better prices than the corresponding real orders in the same contract. This can occur when two or more traders incrementally improve their order prices in the hope of attracting a trade, since combining the small improvements from two or more real orders can result in a big improvement in their combination. In general, advertising implied opportunities at better prices will encourage traders to enter the opposing orders to trade with them. The more implied opportunities that the match engine of an electronic trading system can calculate/derive, the greater this encouragement will be and the more the Exchange will benefit from increased transaction volume. However, identifying implied opportunities may be computationally intensive. In a high performance trading system where low transaction latency is important, it may be important to identify and advertise implied opportunities quickly so as to improve or maintain market participant interest and/or market liquidity.
[0225] For example, two different outright orders may be resting on the books, or be available to trade or match. The orders may be resting because there are no outright orders that match the resting orders. Thus, each of the orders may wait or rest on the books until an appropriate outright counteroffer comes into the exchange or is placed by a user of the exchange. The orders may be for two different contracts that only differ in delivery dates. It should be appreciated that such orders could be represented as a calendar spread order. Instead of waiting for two appropriate outright orders to be placed that would match the two existing or resting orders, the exchange computer system may identify a hypothetical spread order that, if entered into the system as a tradable spread order, would allow the exchange computer system to match the two outright orders. The exchange may thus advertise or make available a spread order to users of the exchange system that, if matched with a tradable spread order, would allow the exchange to also match the two resting orders. Thus, the match engine is configured to detect that the two resting orders may be combined into an order in the spread instrument and accordingly creates an implied order.
[0226] In other words, the exchange's matching system may imply the counteroffer order by using multiple orders to create the counteroffer order. Examples of spreads include implied IN, implied OUT, 2nd- or multiple-generation, crack spreads, straddle, strangle, butterfly, and pack spreads. Implied IN spread orders are derived from existing outright orders in individual legs. Implied OUT outright orders are derived from a combination of an existing spread order and an existing outright order in one of the individual underlying legs. Implied orders can fill in gaps in the market and allow spreads and outright futures traders to trade in a product where there would otherwise have been little or no available bids and asks.
[0227] For example, implied IN spreads may be created from existing outright orders in individual contracts where an outright order in a spread can be matched with other outright orders in the spread or with a combination of orders in the legs of the spread. An implied OUT spread may be created from the combination of an existing outright order in a spread and an existing outright order in one of the individual underlying leg. An implied IN or implied OUT spread may be created when an electronic match system simultaneously works synthetic spread orders in spread markets and synthetic orders in the individual leg markets without the risk to the trader/broker of being double filled or filled on one leg and not on the other leg.
[0228] By linking the spread and outright markets, implied spread trading increases market liquidity. For example, a buy in one contract month and an offer in another contract month in the same futures contract can create an implied market in the corresponding calendar spread. An

exchange may match an order for a spread product with another order for the spread product. Some existing exchanges attempt to match orders for spread products with multiple orders for legs of the spread products. With such systems, every spread product contract is broken down into a collection of legs and an attempt is made to match orders for the legs.

[0229] Implied orders, unlike real orders, are generated by electronic trading systems. In other words, implied orders are computer generated orders derived from real orders. The system creates the “derived” or “implied” order and provides the implied order as a market that may be traded against. If a trader trades against this implied order, then the real orders that combined to create the implied order and the resulting market are executed as matched trades. Implied orders generally increase overall market liquidity. The creation of implied orders increases the number of tradable items, which has the potential of attracting additional traders. Exchanges benefit from increased transaction volume. Transaction volume may also increase as the number of matched trade items increases.

[0230] Examples of implied spread trading include those disclosed in U.S. Patent Publication No. 2005/0203826, entitled “Implied Spread Trading System,” the entire disclosure of which is incorporated by reference herein and relied upon. Examples of implied markets include those disclosed in U.S. Pat. No. 7,039,610, entitled “Implied Market Trading System,” the entire disclosure of which is incorporated by reference herein and relied upon.

[0231] In some cases, the outright market for the deferred month or other constituent contract may not be sufficiently active to provide market data (e.g., bid-offer data) and/or trade data. Spread instruments involving such contracts may nonetheless be made available by the exchange. The market data from the spread instruments may then be used to determine a settlement price for the constituent contract. The settlement price may be determined, for example, through a boundary constraint-based technique based on the market data (e.g., bid-offer data) for the spread instrument, as described in U.S. Patent Publication No. 2015/0073962 entitled “Boundary Constraint-Based Settlement in Spread Markets” (“the '962 Publication”), the entire disclosure of which is incorporated by reference herein and relied upon. Settlement price determination techniques may be implemented to cover calendar month spread instruments having different deferred month contracts.

Order Book Object Data Structures

[0232] In one embodiment, the messages and/or values received for each object may be stored in queues according to value and/or priority techniques implemented by an exchange computing system **100**. FIG. **3**A illustrates an example data structure **300**, which may be stored in a memory or other storage device, such as the memory **204** or storage device **206** described with respect to FIG. **2**, for storing and retrieving messages related to different values for the same action for an object. For example, data structure **300** may be a set of queues or linked lists for multiple values for an action, e.g., bid, on an object. Data structure **300** may be implemented as a database. It should be appreciated that the system may store multiple values for the same action for an object, for example, because multiple users submitted messages to buy specified quantities of an object at different values. Thus, in one embodiment, the exchange computing system may keep track of different orders or messages for buying or selling quantities of objects at specified values.

[0233] Although the application contemplates using queue data structures for storing messages in a memory, the implementation may involve additional pointers, i.e., memory address pointers, or linking to other data structures. Incoming messages may be stored at an identifiable memory address. The transaction processor can traverse messages in order by pointing to and retrieving different messages from the different memories. Thus, messages that may be depicted sequentially, e.g., in FIG. **3**B below, may actually be stored in memory in disparate locations. The software programs implementing the transaction processing may retrieve and process messages in sequence from the various disparate (e.g., random) locations. Thus, in one embodiment, each queue may store different values, which could represent prices, where each value points to or is linked to the

messages (which may themselves be stored in queues and sequenced according to priority techniques, such as prioritizing by value) that will match at that value. For example, as shown in FIG. **3**A, all of the values relevant to executing an action at different values for an object are stored in a queue. Each value in turn points to, e.g., a linked list or queue logically associated with the values. The linked list stores the messages that instruct the exchange computing system to buy specified quantities of the object at the corresponding value.

[0234] The sequence of the messages in the message queues connected to each value may be determined by exchange implemented priority techniques. For example, in FIG. **3**A, messages M1, M2, M3 and M4 are associated with performing an action (e.g., buying or selling) a certain number of units (may be different for each message) at Value 1. M1 has priority over M2, which has priority over M3, which has priority over M4. Thus, if a counter order matches at Value 1, the system fills as much quantity as possible associated with M1 first, then M2, then M3, and then M4.

[0235] In the illustrated examples, the values may be stored in sequential order, and the best or lead value for a given queue may be readily retrievable by and/or accessible to the disclosed system. Thus, in one embodiment, the value having the best priority may be illustrated as being in the topmost position in a queue, although the system may be configured to place the best priority message in some other predetermined position. In the example of FIG. **3**A, Value 1 is shown as being the best value or lead value, or the top of the book value, for an example Action.

[0236] FIG. **3**B illustrates an example alternative data structure **350** for storing and retrieving messages and related values. It should be appreciated that matches occur based on values, and so all the messages related to a given value may be prioritized over all other messages related to a different value. As shown in FIG. **3**B, the messages may be stored in one queue and grouped by values according to the hierarchy of the values. The hierarchy of the values may depend on the action to be performed.

[0237] For example, if a queue is a sell queue (e.g., the Action is Sell), the lowest value may be given the best priority and the highest value may be given the lowest priority. Thus, as shown in FIG. **3**B, if Value 1 is lower than Value 2 which is lower than Value 3, Value 1 messages may be prioritized over Value 2, which in turn may be prioritized over Value 3.

[0238] Within Value 1, M1 is prioritized over M2, which in turn is prioritized over M3, which in turn is prioritized over M4. Within Value 2, M5 is prioritized over M6, which in turn is prioritized over M7, which in turn is prioritized over M8. Within Value 3, M9 is prioritized over M10, which in turn is prioritized over M11, which in turn is prioritized over M12.

[0239] Alternatively, the messages may be stored in a tree-node data structure that defines the priorities of the messages. In one embodiment, the messages may make up the nodes.

[0240] In one embodiment, the system may traverse through a number of different values and associated messages when processing an incoming message. Traversing values may involve the processor loading each value, checking that value and deciding whether to load another value, i.e., by accessing the address pointed at by the address pointer value. In particular, referring to FIG. **3**B, if the queue is for selling an object for the listed Values 1, 2 and 3 (where Value 1 is lower than Value 2 which is lower than Value 3), and if the system receives an incoming aggressing order to buy quantity X at a Value 4 that is greater than Values 1, 2, and 3, the system will fill as much of quantity X as possible by first traversing through the messages under Value 1 (in sequence M1, M2, M3, M4). If any of the quantity of X remains, the system traverses down the prioritized queue until all of the incoming order is filled (e.g., all of X is matched) or until all of the quantities of M1 through M12 are filled. Any remaining, unmatched quantity remains on the books, e.g., as a resting order at Value 4, which was the entered value or the message's value.

[0241] The system may traverse the queues and check the values in a queue, and upon finding the appropriate value, may locate the messages involved in making that value available to the system. When an outright message value is stored in a queue, and when that outright message is involved in a trade or match, the system may check the queue for the value, and then may check the data

structure storing messages associated with that value.
[0242] In one embodiment, an exchange computing system may convert all financial instruments to objects. In one embodiment, an object may represent the order book for a financial instrument. Moreover, in one embodiment, an object may be defined by two queues, one queue for each action that can be performed by a user on the object. For example, an order book converted to an object may be represented by an Ask queue and a Bid queue. Resting messages or orders associated with the respective financial instrument may be stored in the appropriate queue and recalled therefrom.
[0243] In one embodiment, the messages associated with objects may be stored in specific ways depending on the characteristics of the various messages and the states of the various objects in memory. For example, a system may hold certain resting messages in queue until the message is to be processed, e.g., the message is involved in a match. The order, sequence or priority given to messages may depend on the characteristics of the message. For example, in certain environments, messages may indicate an action that a computer in the system should perform. Actions may be complementary actions, or require more than one message to complete. For example, a system may be tasked with matching messages or actions contained within messages. The messages that are not matched may be queued by the system in a data queue or other structure, e.g., a data tree having nodes representing messages or orders.
[0244] The queues are structured so that the messages are stored in sequence according to priority. Although the embodiments are disclosed as being implemented in queues, it should be understood that different data structures such as for example linked lists or trees may also be used.
[0245] The system may include separate data structures, e.g., queues, for different actions associated with different objects within the system. For example, in one embodiment, the system may include a queue for each possible action that can be performed on an object. The action may be associated with a value. The system prioritizes the actions based in part on the associated value.
[0246] For example, as shown in FIG. **3**C, the order book module of a computing system may include several paired queues, such as queues Bid and Ask for an object **302** (e.g., Object A). The system may include two queues, or one pair of queues, for each object that is matched or processed by the system. In one embodiment, the system stores messages in the queues that have not yet been matched or processed. FIG. **3**C may be an implementation of the data structures disclosed in FIGS. **3**A and/or **3**B. Each queue may have a top of book, or lead, position, such as positions **304** and **306**, which stores data that is retrievable.
[0247] The queues may define the priority or sequence in which messages are processed upon a match event. For example, two messages stored in a queue may represent performing the same action at the same value. When a third message is received by the system that represents a matching action at the same value, the system may need to select one of the two waiting, or resting, messages as the message to use for a match. Thus, when multiple messages can be matched at the same value, the exchange may have a choice or some flexibility regarding the message that is matched. The queues may define the priority in which orders that are otherwise equivalent (e.g., same action for the same object at the same value) are processed.
[0248] The system may include a pair of queues for each object, e.g., a bid and ask queue for each object. Each queue may be for example implemented utilizing the data structure of FIG. **3**B. The exchange may be able to specify the conditions upon which a message for an object should be placed in a queue. For example, the system may include one queue for each possible action that can be performed on an object. The system may be configured to process messages that match with each other. In one embodiment, a message that indicates performing an action at a value may match with a message indicating performing a corresponding action at the same value. Or, the system may determine the existence of a match when messages for the same value exist in both queues of the same object.
[0249] The messages may be received from the same or different users or traders.
[0250] The queues illustrated in FIG. **3**C hold or store messages received by a computing

exchange, e.g., messages submitted by a user to the computing exchange, and waiting for a proper match. It should be appreciated that the queues may also hold or store implieds, e.g., implied messages generated by the exchange system, such as messages implied in or implied out as described herein. The system thus adds messages to the queues as they are received, e.g., messages submitted by users, or generated, e.g., implied messages generated by the exchanges. The sequence or prioritization of messages in the queues is based on information about the messages and the overall state of the various objects in the system.

[0251] When the data transaction processing system is implemented as an exchange computing system, as discussed above, different client computers submit electronic data transaction request messages to the exchange computing system. Electronic data transaction request messages include requests to perform a transaction on a data object, e.g., at a value for a quantity. The exchange computing system includes a transaction processor, e.g., a hardware matching processor or match engine, that matches, or attempts to match, pairs of messages with each other. For example, messages may match if they contain counter instructions (e.g., one message includes instructions to buy, the other message includes instructions to sell) for the same product at the same value. In some cases, depending on the nature of the message, the value at which a match occurs may be the submitted value or a better value. A better value may mean higher or lower value depending on the specific transaction requested. For example, a buy order may match at the submitted buy value or a lower (e.g., better) value. A sell order may match at the submitted sell value or a higher (e.g., better) value.

Transaction Processor Data Structures

[0252] FIG. **4** illustrates an example embodiment of a data structure used to implement match engine module **106**. Match engine module **106** may include a conversion component **402**, pre-match queue **404**, match component **406**, post-match queue **408** and publish component **410**.

[0253] Although the embodiments are disclosed as being implemented in queues, it should be understood that different data structures, such as for example linked lists or trees, may also be used. Although the application contemplates using queue data structures for storing messages in a memory, the implementation may involve additional pointers, i.e., memory address pointers, or linking to other data structures. Thus, in one embodiment, each incoming message may be stored at an identifiable memory address. The transaction processing components can traverse messages in order by pointing to and retrieving different messages from the different memories. Thus, messages that may be processed sequentially in queues may actually be stored in memory in disparate locations. The software programs implementing the transaction processing may retrieve and process messages in sequence from the various disparate (e.g., random) locations.

[0254] The queues described herein may, in one embodiment, be structured so that the messages are stored in sequence according to time of receipt, e.g., they may be first in first out (FIFO) queues.

[0255] The match engine module **106** may be an example of a transaction processing system. The pre-match queue **404** may be an example of a pre-transaction queue. The match component **406** may be an example of a transaction component. The post-match queue **408** may be an example of a post-transaction queue. The publish component **410** may be an example of a distribution component. The transaction component may process messages and generate transaction component results.

[0256] In one embodiment, the publish component may be a distribution component that can distribute data to one or more market participant computers. In one embodiment, match engine module **106** operates according to a first in, first out (FIFO) ordering. The conversion component **402** converts or extracts a message received from a trader via the Market Segment Gateway or MSG into a message format that can be input into the pre-match queue **404**.

[0257] Messages from the pre-match queue may enter the match component **406** sequentially and may be processed sequentially. In one regard, the pre-transaction queue, e.g., the pre-match queue,

may be considered to be a buffer or waiting spot for messages before they can enter and be processed by the transaction component, e.g., the match component. The match component matches orders, and the time a messages spends being processed by the match component can vary, depending on the contents of the message and resting orders on the book. Thus, newly received messages wait in the pre-transaction queue until the match component is ready to process those messages. Moreover, messages are received and processed sequentially or in a first-in, first-out FIFO methodology. The first message that enters the pre-match or pre-transaction queue will be the first message to exit the pre-match queue and enter the match component. In one embodiment, there is no out-of-order message processing for messages received by the transaction processing system. The pre-match and post-match queues are, in one embodiment, fixed in size, and any messages received when the queues are full may need to wait outside the transaction processing system or be re-sent to the transaction processing system.

[0258] The match component **406** processes an order or message, at which point the transaction processing system may consider the order or message as having been processed. The match component **406** may generate one message or more than one message, depending on whether an incoming order was successfully matched by the match component. An order message that matches against a resting order in the order book may generate dozens or hundreds of messages. For example, a large incoming order may match against several smaller resting orders at the same price level. For example, if many orders match due to a new order message, the match engine needs to send out multiple messages informing traders which resting orders have matched. Or, an order message may not match any resting order and only generate an acknowledgement message. Thus, the match component **406** in one embodiment will generate at least one message, but may generate more messages, depending upon the activities occurring in the match component. For example, the more orders that are matched due to a given message being processed by the match component, the more time may be needed to process that message. Other messages behind that given message will have to wait in the pre-match queue.

[0259] Messages resulting from matches in the match component **406** enter the post-match queue **408**. The post-match queue may be similar in functionality and structure to the pre-match queue discussed above, e.g., the post-match queue is a FIFO queue of fixed size. As illustrated in FIG. **4**, a difference between the pre- and post-match queues may be the location and contents of the structures, namely, the pre-match queue stores messages that are waiting to be processed, whereas the post-match queue stores match component results due to matching by the match component. The match component receives messages from the pre-match queue, and sends match component results to the post-match queue. In one embodiment, the time that results messages, generated due to the transaction processing of a given message, spend in the post-match queue is not included in the latency calculation for the given message.

[0260] Messages from the post-match queue **408** enter the publish component **410** sequentially and are published via the MSG sequentially. Thus, the messages in the post-match queue **408** are an effect or result of the messages that were previously in the pre-match queue **404**. In other words, messages that are in the pre-match queue **404** at any given time will have an impact on or affect the contents of the post-match queue **408**, depending on the events that occur in the match component **406** once the messages in the pre-match queue **404** enter the match component **406**.

[0261] As noted above, the match engine module **106** in one embodiment operates in a first in first out (FIFO) scheme. In other words, the first message that enters the match engine module **106** is the first message that is processed by the match engine module **106**. Thus, the match engine module **106** in one embodiment processes messages in the order the messages are received. In FIG. **4**, as shown by the data flow arrow, data is processed sequentially by the illustrated structures from left to right, beginning at the conversion component **402**, to the pre-match queue, to the match component **406**, to the post-match queue **408**, and to the publish component **410**. The overall transaction processing system operates in a FIFO scheme such that data flows from element **402** to

404 to **406** to **408** to **410**, in that order. If any one of the queues or components of the transaction processing system experiences a delay, that creates a backlog for the structures preceding the delayed structure. For example, if the match or transaction component is undergoing a high processing volume, and if the pre-match or pre-transaction queue is full of messages waiting to enter the match or transaction component, the conversion component may not be able to add any more messages to the pre-match or pre-transaction queue.

[0262] Messages wait in the pre-match queue. The time a message waits in the pre-match queue depends upon how many messages are ahead of that message (i.e., earlier messages), and how much time each of the earlier messages spends being serviced or processed by the match component. Messages also wait in the post-match queue. The time a message waits in the post-match queue depends upon how many messages are ahead of that message (i.e., earlier messages), and how much time each of the earlier messages spends being serviced or processed by the publish component. These wait times may be viewed as a latency that can affect a market participant's trading strategy.

[0263] After a message is published (after being processed by the components and/or queues of the match engine module), e.g., via a market data feed, the message becomes public information and is publically viewable and accessible. Traders consuming such published messages may act upon those message, e.g., submit additional new input messages to the exchange computing system responsive to the published messages.

[0264] The match component attempts to match aggressing or incoming orders against resting orders. If an aggressing order does not match any resting orders, then the aggressing order may become a resting order, or an order resting on the books. For example, if a message includes a new order that is specified to have a one-year time in force, and the new order does not match any existing resting order, the new order will essentially become a resting order to be matched (or attempted to be matched) with some future aggressing order. The new order will then remain on the books for one year. On the other hand, a new order specified as a fill or kill (e.g., if the order cannot be filled or matched with an order currently resting on the books, the order should be canceled) will never become a resting order, because it will either be filled or matched with a currently resting order, or it will be canceled. The amount of time needed to process or service a message once that message has entered the match component may be referred to as a service time. The service time for a message may depend on the state of the order books when the message enters the match component, as well as the contents, e.g., orders, that are in the message.

[0265] In one embodiment, orders in a message are considered to be “locked in”, or processed, or committed, upon reaching and entering the match component. If the terms of the aggressing order match a resting order when the aggressing order enters the match component, then the aggressing order will be in one embodiment guaranteed to match.

[0266] As noted above, the latency experienced by a message, or the amount of time a message spends waiting to enter the match component, depends upon how many messages are ahead of that message (i.e., earlier messages), and how much time each of the earlier messages spends being serviced or processed by the match component. The amount of time a match component spends processing, matching or attempting to match a message depends upon the type of message, or the characteristics of the message. The time spent inside the processor may be considered to be a service time, e.g., the amount of time a message spends being processed or serviced by the processor.

[0267] The number of matches or fills that may be generated in response to a new order message for a financial instrument will depend on the state of the data object representing the electronic marketplace for the financial instrument. The state of the match engine can change based on the contents of incoming messages.

[0268] It should be appreciated that the match engine's overall latency is in part a result of the match engine processing the messages it receives. The match component's service time may be a

function of the message type (e.g., new, modify, cancel), message arrival rate (e.g., how many orders or messages is the match engine module receiving, e.g., messages per second), message arrival time (e.g., the time a message hits the inbound MSG or market segment gateway), number of fills generated (e.g., how many fills were generated due to a given message, or how many orders matched due to an aggressing or received order), or number of Mass Quote entries (e.g., how many of the entries request a mass quote).

[0269] In one embodiment, the time a message spends: [0270] Being converted in the conversion component **402** may be referred to as a conversion time; [0271] Waiting in the pre-match queue **404** may be referred to as a wait until match time; [0272] Being processed or serviced in the match component **406** may be referred to as a matching time; [0273] Waiting in the post-match queue **408** may be referred to as a wait until publish time; and [0274] Being processed or published via the publish component **410** may be referred to as a publishing time.

[0275] It should be appreciated that the latency may be calculated, in one embodiment, as the sum of the conversion time and wait until match time. Or, the system may calculate latency as the sum of the conversion time, wait until match time, matching time, wait until publish time, and publishing time. In systems where some or all of those times are negligible, or consistent, a measured latency may only include the sum of some of those times. Or, a system may be designed to only calculate one of the times that is the most variable, or that dominates (e.g., percentage wise) the overall latency. For example, some market participants may only care about how long a newly sent message that is added to the end of the pre-match queue will spend waiting in the pre-match queue. Other market participants may care about how long that market participant will have to wait to receive an acknowledgement from the match engine that a message has entered the match component. Yet other market participants may care about how much time will pass from when a message is sent to the match engine's conversion component to when match component results exit or egress from the publish component.

Electronic Message Packets/Electronic Message Data Segments

[0276] Packets and packet switched networks are used extensively in electronic communications. Packet switched networks utilize a digital networking communications method that groups all transmitted data, regardless of content, type, or structure, into suitably sized blocks, called packets. Generally, packets may contain control information as well as user data, also known as a payload or actual content. Typically, the control information provides data the network needs to deliver the user data such as source and destination network addresses and the user data involves the actual content intended to be communicated between the source and the destination. For example, packets may be considered messages, with the control information providing addresses or information about the actual content of the message included as user data.

[0277] Under the Open System Interconnection (“OSI”) model, which is a conceptual model that characterizes and standardizes the internal functions of a communication system by partitioning it into abstraction layers, the physical abstraction layer defines electrical and physical specifications for devices. In particular, it defines the relationship between a device and a transmission medium, such as a copper or fiber optical cable. This includes the layout of pins, voltages, line impedance, cable specifications, signal timing, hubs, repeaters, network adapters, host bus adapters (HBA used in storage area networks) and more. The major functions and services performed by the physical layer include: establishment and termination of a connection to a communications medium; participation in the process whereby the communication resources are effectively shared among multiple users, for example, contention resolution and flow control; and modulation or conversion between the representation of digital data in user equipment and the corresponding signals transmitted over a communications channel, these signals operating over the physical cabling (such as copper and optical fiber) or over a radio link.

[0278] In a TCP/IP implementation, a source (e.g., client computer) application may provide application layer data to a TCP layer (transport layer), which divides the data into segments and

adds a TCP header to the segmented data. The TCP layer passes TCP segments to an IP layer, which adds an IP header to the TCP segments and passes the information/data to the recipient in the form of IP packets communicated over the IP protocol. The IP layer relies on a physical layer which transmit the IP packets in network frames to the recipient. Each layer may add its own header to the data received from another layer. On the recipient side, the physical layer passes IP packets to the IP layer, which passes TCP segments to the TCP (transport) layer, which in turn passes application data to the destination application.

[0279] As used herein, electronic message data segments may refer to TCP segments, which may include IP packets. Electronic message packets may refer to IP packets, or may refer generally any division/packeting of data, e.g., for easier/more convenient transmission thereof.

[0280] Electronic message packets may be communicated via networks. Generally, a network interconnects one or more computers so that they may communicate with one another, whether they are in the same room or building (such as a Local Area Network or LAN) or across the country from each other (such as a Wide Area Network or WAN). A network is a series of points or nodes interconnected by communications paths. Networks can interconnect with other networks and can contain sub-networks. A node is a connection point, either a redistribution point or an end point, for data transmissions generated between the computers which are connected to the network. In general, a node has a programmed or engineered capability to recognize and process or forward transmissions to other nodes. The nodes can be computer workstations, servers, bridges or other devices but typically, these nodes are routers or switches. Electronic message packets may be communicated from an origin through a series of nodes to an intended final destination.

[0281] Further, even as electronic message packets arrive at a destination (e.g., the data transaction processing system), the handling of the electronic message packets at the destination may also involve multiple steps, component interactions, and processes until the message is ultimately received by a destination application for use thereby. This process may be further complicated if multiple electronic message packets from multiple origins are communicated to a common destination application using communication protocols that organize electronic message packets based on origin and/or prioritization rules for processing at the common destination. For example, a communication protocol may indicate that a buffer will be created for each source at the destination, and that electronic message packets from each source will be placed in the respective buffer for each source at the destination. The receiving computer may reassemble the electronic message packets into electronic message data segments, which may in turn be reassembled and provided to a destination application. The destination application may process, consume, or otherwise use the electronic message packet or electronic message data segment payload, and report on the results of the processing, consumption, and/or usage of the messages back to the origin or source of the message. For example, the electronic message packets and/or electronic message data segments may be provided to an order entry gateway in a deterministic manner. For additional information on deterministic processing, see U.S. Pub. No. 2015/0178831, filed on Dec. 19, 2013, entitled “Deterministic And Efficient Message Packet Management”, the entirety of which is incorporated by reference herein and relied upon. The order entry gateway may be configured to collect the electronic message packets/segments and provide them to an order book or a match engine for one or more financial instruments. A match engine may then use the electronic message packets and the determined order to match orders represented by the electronic message packets based on a priority determined from the determined order. The communications protocol may provide a transport layer acknowledgement message to the source of an electronic message data segment substantially immediately, e.g., before the electronic message data segment is processed by a destination application. In the process, it may be necessary for the data transaction processing system to generate and associate a unique identification number for each electronic data transaction request message, and provide the identification number to the message source associated with the electronic data transaction request message. In one embodiment, the

message source (e.g., client computer) cannot submit additional instructions (e.g., additional electronic data transaction request messages) associated with an initial electronic data transaction request message until the identification number of the initial electronic data transaction request message is provided to the client computer. The disclosed embodiments facilitate efficient processing of electronic message data segments communicated to an application via a network from a plurality of message sources using a communications protocol which generates a transport layer acknowledgement message upon receipt of an electronic message data segment.

[0282] FIG. **5** illustrates an example computer system **500** including an example embodiment of message management module **116**. Message management module **116** includes a receiver **502**, clock **504**, identification number generator **506**, and transmitter **608**. Message management module **116** also includes a processor and a memory coupled therewith (not shown) which may be implemented as a processor **202** and memory **204** as described with respect to FIG. **2**. Message management module **116** may implement a transport layer protocol, such as TCP.

[0283] Client computer **530** sends data, e.g., electronic data transaction request messages segmented into electronic message data segments and/or packeted into electronic message packets, to, and receives data from, message management module **116** over connection **512**. Connection **512** may be made over network **162**. In one embodiment, connection **512** is a TCP (Transmission Control Protocol) connections, implemented by TCP functionality, also referred to as a TCP stack, at both the transmitting and receiving devices, which have a built-in acknowledgment feature that enables the client computer **530** to receive an acknowledgment, or ACK (e.g., transport layer acknowledgement message), to indicate that message management module **116** received an electronic data transaction request message (which may be segmented and/or packeted) from client computer **530**.

[0284] As is known in the art, the first phase of a TCP session is establishment of the connection. This requires a three-way handshake, referred to as SYN-SYN-ACK. Once a TCP session between two connections is established, typical TCP acknowledgment behavior is to acknowledge, via an ACK, the highest sequence number of in-order bytes received. TCP provides for options that alter default acknowledgment behavior, such as SACK (Selective Acknowledgment) which allows the receiver to modify the acknowledgment field to describe noncontinuous blocks of received data, so that the sender can retransmit only what is missing at the receiver's end. Accordingly, the acknowledgment serves to indicate how much of the previously transmitted data was successfully received. That is, where acknowledgment of receipt of previously transmitted data is not received, that data may be retransmitted, such as after a configurable period of time has elapsed. The TCP protocol may provide for multiple attempts to successfully transmit data before ceasing further attempts. Upon successful transmission of data, the TCP protocol may provide for increasing the amount of data sent in a given transmission, i.e. by increasing the size of a receiving window.

[0285] Receiver **502** receives electronic data transaction request messages from client computer **530**. Receiver **502** may implement TCP/IP (Transmission Control Protocol/Internet Protocol), and may be configured to receive and extract TCP level information. Client computer **530** may transmit electronic data transaction request messages in one or more electronic message data segments, as discussed herein.

[0286] In one embodiment, receiver **502** may be referred to as a transaction receiver or orderer, and may correspond to the transaction receiver or orderer described in U.S. patent application Ser. No. 15/232,224, filed on Aug. 9, 2016, entitled “Systems and Methods for Coordinating Processing of Instructions Across Multiple Components” (“the '224 Application”), the entirety of which is incorporated by reference herein and relied upon. As described in the '224 Application, the exchange computing system may be configured to detect the time signal data associated with incoming transactions, or data indicative of a time of receipt of the transaction. Receiver **502** may augment each electronic data transaction request message or transaction request with time signal data, or data indicative of a time of receipt or time or sequence indicative of a temporal or

sequential relationship between a received transaction request and other received transaction requests. The time signal data may be based on, for example, clock **504**, described below.
[0287] The sequenced/augmented transaction requests may then be substantially simultaneously communicated, e.g. broadcasted, to the match engine module **106** which then processes the electronic data transaction request message, based on the sequencing imparted by receiver **502**, and determines a result, referred to as a match event, indicative, for example, of whether the order to trade was matched with a prior order, or reflective of some other change in a state of an electronic marketplace, etc. As used herein, match events generally refer to information, messages, alerts, signals or other indicators, which may be electronically or otherwise transmitted or communicated, indicative of a status of, or updates/changes to, a market/order book, i.e. one or more databases/data structures which store and/or maintain data indicative of a market for, e.g. current offers to buy and sell, a financial product, described in more detail below, or the match engines associated therewith.
[0288] Receiver **502** may be implemented as one or more logic components such as on an FPGA which may include a memory or reconfigurable component to store logic and processing component to execute the stored logic, coupled with the network **162**, such as via the interconnection infrastructure **202**, and operative to receive each of the plurality of financial transactions and, upon receipt, augment, or otherwise ascribe or associate with, the received financial transaction with time signal data, which may be based on a system clock of system **100**, or a system clock associated with transaction receiver **210**, such as clock **504**, or any type of sequential counter.
[0289] In one embodiment, receiver **502** and/or transmitter **510** may be implemented in a network interface controller ("NIC") so as to receive, process and respond to electronic data transaction request messages as close to a network connection/interface as possible, minimizing any intervening hardware and software components and the associated latency introduced thereby. In particular, the receiver **502** and/or transmitter **510** may be implemented in a "Smart NIC" which incorporates a field programmable gate array ("FPGA") or other programmable logic to provide processing capability to be collocated, such as via a custom interface, or otherwise closely interconnected with networking equipment, such as a router or switch, e.g. via a backplane thereof. This would allow for access to incoming electronic data transaction request messages as quickly as possible and avoid some of the latency introduced, not only by having to route the transaction over conventional networking media, but also by the communications protocols, e.g. Transport Control Protocol ("TCP"), used to perform that routing. One exemplary implementation is referred to as a "Smart Network Interface Controller" or "SmartNIC" which is a device which typically brings together high-speed network interfaces, a PCIe host interface, memory and an FPGA. The FPGA implements the NIC controller, acting as the bridge between the host computer and the network at the "physical layer" and allows user-designed custom processing logic to be integrated directly into the data path.
[0290] Clock **504** may be a hardware unit, such as the Solarflare Precision Time Protocol (PTP)™ hardware. Clock **504** provides a single source of time, which, as described herein, is used to augment electronic message data segment with time signal data, and to generate identification numbers for electronic data transaction request messages contained in electronic message data segments.
[0291] Alternatively, clock **504** may be replaced by a counter (not shown) that associates increasing numbers to each incoming electronic message data segment or electronic data transaction request message. The number assigned by the counter may then be used to generate an identification number as described herein. Because many data transaction processing systems already include a clock or a time-stamping device, it may be more efficient to use clock signal data to generate the identification number. Moreover, by using a clock, the identification number then can indicate, to the client computer, the time of receipt of an electronic data transaction request message. This information may be valuable for the client computers.

[0292] Receiver **502** may also be operative to augment, or otherwise ascribe or associate with, the received financial transactions (e.g., electronic data transaction request messages) with sequence data, such as an ordering or sequence number, indicative of a relationship, temporal or otherwise based on business rules/logic, e.g. a deterministic relationship, between the received financial transaction, e.g. the time of receipt thereof, and any of the plurality of financial transactions, e.g. the times of receipt thereof, previously and/or subsequently received by the receiver **502**. The ascribed ordering may then implicitly define the relationship with those transaction requests received thereafter. In one embodiment, the ordering may be a time stamp or, alternatively, an incremented sequence number. The receiver may augment each incoming electronic message data segment with time information received from clock **504**.
[0293] It should be appreciated that if the receiver **502** augments the received electronic data transaction request messages/financial transactions with time signal data and timestamp data, which may be the same data, the data may be used in different ways, as described herein. For example, a timestamp may be used by the transaction receiver to determine an order in which the financial transactions are processed by one or more match engine modules downstream.
[0294] In one embodiment, the receiver **502** may augment electronic data transaction request messages with time signal data immediately or in real time, e.g., upon receipt of the electronic data transaction request message.
[0295] Identification number generator **506** generates a unique identification number for each separate electronic data transaction request message based on a timestamp assigned, e.g., by clock **504**, upon receiving the electronic message data segment containing the electronic data transaction request message. The data transaction processing system assigns a unique identification number to each transaction request submitted by a client computer. The identification number allows a client computer to refer to a previously submitted electronic data transaction request message.
[0296] In one embodiment, once an identification number is generated, it is associated with an electronic data transaction request message through the life of the message as it is processed by other components of the exchange computing system, such as match engine module **106** and order book module **110**. Alternatively, the exchange computing system may assign a new identification number each time an order is modified. For example, an order may be associated with identification number ID1. If the client computer modifies the order, the exchange computing system may assign identification number ID2 to the order. The client computer and the exchange computing system then refer to the order as ID2.
[0297] The exchange computing system may disseminate the identification number for an electronic data transaction request message to the respective message source, e.g., the client computer that submitted the electronic data transaction request message. Thus, the exchange computing system and client computers use the identification number to refer to an electronic data transaction request message. Each electronic data transaction request message may have or be associated with a unique identification number. The exchange computing system may implement rules to which each valid identification number must adhere. For example, the exchange computing system may require that the identification number monotonically increase as new messages are received. Identification numbers may need to be in a specified format, and may need to be a certain fixed length.
[0298] Identification numbers may be useful for client computers to control previously submitted orders. For example, a trader who submits a first order to purchase a quantity of a financial instrument may change his/her strategy and may wish to cancel or modify his/her first order. The trader needs the identification number of the first order to be able to send an electronic data transaction request message modifying the first order. Without the identification number of the first order, the trader cannot control any aspect of the first order.
[0299] Some data transaction processing systems may allow client computers to generate and submit their own identification numbers for their electronic data transaction request messages. This

strategy ensures that client computers always know the identification number associated with all of their orders. However, this strategy requires the exchange computing system to then validate/check that each identification number generated by the client computers follows the prescribed rules for identification numbers. For example, the exchange computing system may then need to check that each identification number is unique within the exchange computing system, creating extra processing work for the exchange computing system, which requires the exchange computing system to expend additional computing resources and time. By generating identification numbers for electronic data transaction request messages, the exchange computing system can then associate each identification number with the electronic data transaction request message for the life of the order, e.g., until the order remains (e.g., resting) in the exchange computing system.

[0300] Injector **508** injects the identification number into a transport layer acknowledgement message generated by the transmitter **510**. Transmitter **510** transmits electronic data transaction result messages to client computer **530**. Transmitter **510** may implement TCP/IP (Transmission Control Protocol/Internet Protocol), and may be configured to write and transmit TCP level information in acknowledgement messages. In one embodiment, the injector **508** adds identification number data, received from identification number generator **506**, to TCP level acknowledgments transmitted by transmitter **510** to the client computer **530**, the source of the electronic message data segments acknowledged by the transmitter **510**.

[0301] Injector **508** and transmitter **510** may be configured to modify and transmit the transport layer acknowledgement message to client computer **530** before the corresponding electronic data transaction request message is communicated to a destination application, e.g., the match engine module or order book module, or a gateway application leading to the match engine module or order book module.

[0302] Receiver **502** and/or transmitter **510** may be integrated with the TCP implementation on the host computer system, e.g., exchange computing system **100**, to receive incoming electronic data transaction request messages from client computer **530** and to transmit responses, electronic data transaction result messages, and to receive and transmit TCP level information as described in further detail below. As is known in the art, TCP implementations may be configured to send a TCP acknowledgment to client computers acknowledging receipt of the electronic data transaction request message. In one implementation, receiver **502** and/or transmitter **510** may be integrated with a customized version of the TCP protocol stack and may be further implemented within the NIC to perform the functions described below.

[0303] In one embodiment, the disclosed message management module **116** modifies the TCP protocol to add or inject hardware based information in the TCP ACK/transport layer acknowledgement message sent substantially immediately to the client computer **530**. In one embodiment, the hardware based information may be an identification number that is associated with each electronic data transaction request message for the life of the electronic data transaction request message, i.e., for as long as the electronic data transaction request message is maintained and stored within the exchange computing system **100**.

[0304] In one embodiment, after receipt of an electronic data transaction request message by receiver **502**, transmitter **510** transmits a hardware based identification number within TCP to the client computer **530** acknowledging receipt of the electronic data transaction request message.

[0305] After the electronic data transaction request message is processed by the match engine module **106**, the message management module **116** may again transmit the hardware based identification number to the client computer in an electronic data transaction result message that contains information relating to the requested transaction.

[0306] FIG. **6** illustrates an example electronic message data segment **600**, such as a TCP segment, transmitted by client computer **530** to message management module **116**. Electronic message data segment **600** is received by receiver **502**. The electronic message data segment may include multiple electronic data transaction request messages, such as M1, M2, M3, M4. The electronic

message data segment **600** may associate each electronic data transaction request message in an ordinal position. For example, M1 is associated with ordinal position Pos1, M2 is associated with ordinal position Pos2, M3 is associated with ordinal position Pos3, and M4 is associated with ordinal position Pos4. Client computer **530** generates a plurality of electronic data transaction request messages, and generates electronic message data segments that are transmitted, e.g., by further generating electronic message packets, over the network to the exchange computing system.

[0307] In an embodiment, one electronic data transaction request message may span multiple electronic message data segments. For example, client computer **530** prepares electronic message data segments that will convey electronic data transaction request messages to the data transaction processing system **100**. Electronic message data segments may be of a fixed size, or may be variable but have a maximum size. If an electronic data transaction request message does not fit into an existing electronic message data segment, the client computer may add the remaining portion of the electronic data transaction request message into the next electronic message data segment. TCP/IP works to create appropriate segments and packets as needed for transmission over connection **512**. If an electronic data transaction request message spans two electronic message data segments, the electronic data transaction request message is considered to be received (and accordingly time stamped) by the data transaction processing system when the later of the two electronic message data segments is received by the data transaction processing system. In other words, an electronic data transaction request message is received by the data transaction processing system, and associated with a time stamp, when the last part of the electronic data transaction request message is received by the data transaction processing system. Alternatively, an electronic data transaction request message may be considered to be received, and associated with a time stamp, when the first part of the electronic data transaction request message is received by the data transaction processing system.

[0308] For example, if an order submitted by a trader spans multiple IP packets/TCP segments, the message management module may use the latest timestamp/timestamp from the last packet received to generate the identification number. The previous packets may not receive an identification number in their ACKs, or TCP ACKs without an identification number for an electronic data transaction request message may be delayed using the TCP protocol's delay features.

[0309] The exchange computing system may be able to determine that the last packet/segment for an electronic data transaction request message has been received based on closing fields of the TCP/IP header fields that indicate when all the data for a segment or packet have been received.

[0310] It should be appreciated that the receiver **502** may handle other messages that do not include orders, or transaction requests. The message management module may be configured to distinguish between electronic message data segment that include orders, e.g., electronic data transaction request messages, and electronic message data segment that do not include orders, and only generate an identification number if an electronic message data segment includes at least one electronic data transaction request message. Alternatively, the message management module may generate identification numbers for all electronic message data segments received, regardless of whether the electronic message data segment includes an order or not. The client computer can then just ignore identification numbers included in a transport layer acknowledgement message for an electronic message data segment that did not include an electronic data transaction request message.

[0311] FIG. **7** illustrates an example system **700** including a plurality of client computers **530** and **532** transmitting messages to exchange computing system **100**, including message management module **116**, over TCP/IP. Per the TCP/IP protocol, each client computer maintains a TCP connection/session with exchange computing system **100**. In particular, client computer **530** communicates with message management module **116** over TCP session **512**, and client computer **532** communicates with message management module **116** over TCP session **514**. Client computer

530 transmits electronic message data segments **702**, **704** and **706** to message management module **116**. Client computer **532** transmits electronic message data segments **714** and **716** to message management module **116**. The TCP protocol, implemented by receiver **502**/transmitter **510**, generates transport layer acknowledgement messages for each received electronic message data segment. Transmitter **510** transmits transport layer acknowledgement messages **708**, **710** and **712** acknowledging that electronic message data segments **702**, **704** and **706**, respectively, have been received by receiver **502**. Transmitter **510** transmits transport layer acknowledgement messages **718** and **720** acknowledging that electronic message data segments **714** and **716**, respectively, have been received by receiver **502**. TCP guarantees that segments are received by message management module **116** in the order they are transmitted by the client computers.

[0312] The exchange computing system generates and provides to each client computer an identification number for each electronic data transaction request message. However, as described in connection with FIG. **6**, an electronic message data segment may include multiple electronic data transaction request messages. The disclosed embodiments generate identification numbers for each electronic data transaction request message, but transmit only one identification number in a transport layer acknowledgement message responsive to an electronic message data segment, where one electronic message data segment may include multiple electronic data transaction request messages.

[0313] Alternatively, the transport layer acknowledgement message responsive to an electronic message data segment includes the identification number for each electronic data transaction request message contained in the electronic message data segment.

[0314] FIG. **8** illustrates an example computer implemented method for facilitation of efficient processing of a plurality of electronic message data segments communicated to an application via a network from a plurality of message sources using a communications protocol which generates a transport layer acknowledgement message upon receipt of an electronic message data segment, which may be implemented with computer devices and computer networks, such as those described with respect to FIG. **2**. Embodiments may involve all, more or fewer actions indicated by the actions of FIG. **8**. The actions may be performed in the order or sequence shown or in a different sequence.

[0315] At step **802**, a network interface coupled with a network receives a plurality of electronic message data segments from the network. Each electronic message data segment includes one or more electronic data transaction request messages, and each electronic data transaction request message in an electronic message data segment is associated with an ordinal position of the electronic message data segment. For example, an electronic message data segment may include four different positions, or ordinal positions, that can hold electronic data transaction request messages.

[0316] If an electronic data transaction request message spans multiple electronic message data segments, the last electronic message data segment of the multiple electronic message data segments that includes the last portion of the electronic data transaction request message may be considered to include the electronic data transaction request message.

[0317] Receiving may be performed using any technique operable to acquire or remove the messages from a path through the network upon which the messages are traveling. In an embodiment, the messages are received at the common destination using a network interface controller (“NIC”). Any NIC may be used, however in an embodiment, a NIC may be adapted to implement embodiments disclosed herein using various hardware or software components and configurations, such as those disclosed with respect to FIG. **2**. For example, a NIC may involve multiple (i.e. four) Small Form-Factor Pluggable transceivers (“SFP”) to interface with a network, a Digital Signal Processor (“DSP”) to transform signals from the network into data, an Altera Stratix® series Field Programmable Gate Array (“FPGA”) to perform operations with/on the data, NIC specific Random Access Memory (“RAM”) sized at 48 Gigabytes, a NIC specific Precision

Time Protocol (“PTP”) clock, and a Peripheral Component Interconnect port to connect the NIC to a server or other computer.

[0318] At step **804**, a processor of the network interface associates each received electronic message data segment with time signal data indicative of a time the electronic message data segment was received by the network interface. The time signal data may be received from a clock integrated with the network interface.

[0319] At step **806**, the method includes, for each electronic data transaction request message, generating an identification number based on (i) the time signal data associated with the electronic message data segment associated with the electronic data transaction request message and (ii) the ordinal position associated with the electronic data transaction request message. It should be appreciated that because the identification number is generated based on the time signal data of when an electronic message data segment was received, the identification number cannot be generated until after the electronic message data segment is actually received by the data transaction processing system. For example, the identification number for an electronic data transaction request message cannot be generated ahead of receiving the electronic data transaction request message. Thus, identification number data cannot be pre-generated and stored in a buffer that holds data to be transmitted. As discussed below, the identification number generated after the electronic data transaction request message is received is transmitted to the message source in the transport layer acknowledgement message.

[0320] In one embodiment, the message management module generates a unique identification number for each received electronic data transaction request message. As discussed above, the exchange computing system attempts to process electronic data transaction request messages as quickly as possible. The message management module may accordingly be configured to minimize the complexity and time needed for processing routines, e.g., in hardware or software, that generate an identification number, such that generating identification numbers is performed quickly and without causing additional delays.

[0321] The speed with which an exchange computing system can receive and process electronic data transaction request messages is important to the success of the exchange computing system. It may be important for the data transaction processing system to process and respond to customers’ electronic data transaction request messages as soon as possible. Thus, it may not be desirable to delay processing of electronic data transaction request messages by generating identification numbers before the electronic data transaction request messages are processed (e.g., matched). Moreover, once an identification number is generated, it must be associated with the electronic data transaction request message for the life of the electronic data transaction request message in the exchange computing system. When an identification number is associated with the electronic data transaction request message, the size of the electronic data transaction request message may increase (because it now includes the identification number). Or, the identification number may be stored in a memory, and a pointer to that memory location is then associated with the electronic data transaction request message. Accordingly, to avoid delay of generating identification numbers and complexity of associating identification numbers with electronic data transaction request messages, an exchange computing system may not generate identification numbers until after an electronic data transaction request message enters the match engine (e.g., match component) and has been processed by the match engine. However, this causes customers to have to wait until an electronic data transaction request message is processed to receive its identification number. During this time period, which can vary depending on the state of the exchange computing system and the results of match attempts, customers have no control over their previously submitted orders. For more information on the impact of latency on customers in a data transaction processing system, see U.S. application Ser. No. 15/260,707, entitled “Message Cancelation Based On Data Transaction Processing System Latency”, filed Sep. 9, 2016, the entirety of which is incorporated by reference herein and relied upon. As discussed above, identification numbers provide a useful

way for traders to control previously submitted orders.
[0322] In one embodiment, the disclosed message management module uses hardware level information that is already present and generated in the system (e.g., a timestamp) to quickly and efficiently generate identification numbers without slowing down or delaying processing of electronic data transaction request messages by the exchange computing system. Because the disclosed embodiments allow for rapid generation and TCP level transmission of valid identification numbers, the exchange computing system can balance customer control over messages (via immediately received identification numbers) with the exchange computing system's desire to process electronic data transaction request messages in the match engine as quickly as possible.
[0323] The identification number generator **506** may receive, from the network interface/receiver **502**, time signal data associated with a received electronic message data segment. The identification number generator **506** may truncate some or all of the time signal data to ensure that the injector **508** can add the resulting identification number to the transport layer acknowledgement message. For instance, injector **508** may add the identification number to the TCP options field of a transport layer acknowledgement message. The identification number generator **506** determines the electronic data transaction request messages and their ordinal positions associated with an electronic message data segment. For example, referring to FIG. **6**, the identification number generator **506** may determine that electronic message data segment **600** includes electronic data transaction request messages M1, M2, M3 and M4 at ordinal positions Pos1, Pos2, Pos3 and Pos4, respectively. In one embodiment, the timestamping device, e.g., timecard, implemented by the exchange computing system is selected, or configured, to have a resolution at least as small as the fastest rate at which electronic message data segments are received by the receiver **502**. This ensures that the timestamp for any one electronic message data segment is unique. The identification number generator **506** combines timestamp information with ordinal position information to generate a unique identification number for each electronic data transaction request message.
[0324] In one embodiment, the timestamp generated by the clock **504** may be 64 bits. The identification number generator **506** truncates the higher 32 bits, thus retaining only the lower 32 bits, and adds ordinal position information to the truncated 32 timestamp bits to generate the identification number for electronic message data segment **600**. For example, the identification number generator **506** may generate only one identification number for message M4 in the highest ordinal position in electronic message data segment **600**. If electronic message data segment is received by receiver **502** at timestamp T1, identification number generator **506** may combine bit information representing some or all of T1 with bit information representing Pos4 to generate the identification number for electronic data transaction request message M4.
[0325] In one embodiment, the identification number generator **506** generates identification numbers for M1 through M4, but only shares M4 with the sending client computer via the transport layer acknowledgement message, as discussed below. Thus, the disclosed embodiments reduce the amount of data that is transmitted to the client computer because the client computer does not need to receive identification numbers for all of the electronic data transaction request messages in electronic message data segment **600**.
[0326] The exchange computing system shares the manner/logic in which identification numbers are generated by the identification number generator **506** with the client computers. Thus, client computers can, upon receipt of an identification number as discussed below, determine the identification numbers of other electronic data transaction request messages in an electronic message data segment. For instance, identification number generator **506** sends the identification number for electronic data transaction request message M4 to the sender. The sending client computer created electronic message data segment **600**, and thus knows that electronic data transaction request message M4 is associated with Pos4. The client computer can then use the

information associated with timestamp T1 and with bit information representing Pos1, Pos2, and Pos3 to generate the identification numbers for electronic data transaction request messages M1, M2, and M3, respectively. The disclosed embodiments allow the data transaction processing system to send the identification number for electronic data transaction request message M4 only, but client computer is able to determine the identification numbers for electronic data transaction request messages M1, M2 and M3.

[0327] At step **808**, the method includes, for each electronic data transaction request message, augmenting each electronic data transaction request message with the identification number. At step **810**, the method includes, for each electronic data transaction request message, communicating the augmented electronic data transaction request message to the application. The application may cause the data transaction processing system to match, or attempt to match, the electronic data transaction request message with previously received by unsatisfied electronic data transaction request messages.

[0328] At step **812**, the method includes, for each electronic message data segment, augmenting a transport layer acknowledgement message generated based on the communications protocol acknowledging receipt of the electronic message data segment to include the identification number associated with one of the electronic data transaction request messages associated with the electronic message data segment. In one embodiment, the communications protocol may be configured to generate the transport layer acknowledgement message substantially immediately, or as soon as physically possible for the computing system. In one embodiment, the communications protocol may be TCP which is configured to generate a transport layer acknowledgement message immediately upon receipt of a packet. Injector **508** may be integrated with the network interface and the transport layer such that injector **508** injects or adds the identification number into the transport layer acknowledgement message before the transport layer acknowledgement message is transmitted to the message source.

[0329] The transport layer acknowledgement message may be a TCP ACK. As is known in the art, a TCP segment header may include multiple fields, including a [0330] source TCP port number (2 bytes); [0331] destination TCP port number (2 bytes); [0332] sequence number (4 bytes); [0333] acknowledgement number (4 bytes); [0334] TCP data offset (4 bits); [0335] reserved data (3 bits); [0336] control flags (up to 9 bits); [0337] window size (2 bytes); [0338] TCP checksum (2 bytes); [0339] urgent pointer (2 bytes); and [0340] TCP optional data (0-40 bytes).

[0341] TCP inserts header fields into the message stream in the order listed above.

[0342] In one embodiment, the injector **508** adds the identification number to the TCP Options field in a TCP ACK, or TCP acknowledgement message.

[0343] Injector **508** is configured to insert an identification number in the TCP optional data field. TCP allows piggybacking ACKs onto data packets sent from a receiver to a sender. However, waiting until data is generated, and adding an ACK to that data would cause undesirable delays in acknowledging that a message has been received. “Making Better Use of All Those TCP ACK Packets” by Shang and Wills (“Shang”) describes piggybacking data onto packets carrying ACK information. However, Shang describes buffering and holding data until an ACK is generated. Shang also describes checking if the original sender's buffer can contain the data to that is piggybacked onto the ACK. The disclosed embodiments generate the data to be transmitted, i.e., an identification number conforming to/valid accordingly to the exchange's identification number rules, after the ACK-triggering electronic message data segment is received. The disclosed embodiments modify the TCP header as disclosed herein, and the client computer simply needs to read the header.

[0344] At step **814**, the method includes, for each electronic message data segment, communicating the augmented transport layer acknowledgement message to the message source associated with the electronic message data segment. The client computer may then extract the identification number from the augmented transport layer acknowledgement message, and submit another electronic data

transaction request message including the identification number to modify a previously submitted electronic data transaction request message. For example, as discussed above, a client computer that submitted electronic message data segment **600** may receive the identification number for electronic data transaction request message M4 in a TCP acknowledgement message acknowledging receipt of electronic message data segment **600**. The client computer can then generate the identification numbers for electronic data transaction request messages M1, M2 and M3 based on the identification number received for electronic data transaction request message M4.

[0345] In an embodiment, the method of FIG. **8** includes communicating an augmented transport layer acknowledgement message acknowledging receipt of the electronic message data segment to the message source before any of the augmented electronic data transaction request messages corresponding to the electronic message data segment are processed by the application.

[0346] In an embodiment, the method of FIG. **8** includes receiving, by the network interface coupled with the network, an electronic message data segment including an electronic data transaction request message including an identification number associated with one of the electronic data transaction request messages previously received by the network interface; and modifying, by the application, the previously received electronic data transaction request message based on the electronic data transaction request message.

[0347] In an embodiment, the method of FIG. **8** includes receiving, by a message source, an augmented transport layer acknowledgement message acknowledging receipt of an electronic message data segment, the augmented transport layer acknowledgement message including an identification number for one of the electronic data transaction request messages associated with the electronic message data segment; and generating, by the message source, an identification number for another of the electronic data transaction request messages associated with the electronic message data segment based on the identification number of the one of the electronic data transaction request messages.

[0348] As discussed above, TCP guarantees that packets/segments sent from the sender to the receiver are transmitted in the same sequence. Thus, client computers who send a stream of data and receive a stream of data responsive to the sent stream can easily correlate sent messages with transport layer acknowledgement messages responsive to the sent messages.

[0349] FIG. **9** illustrates an example timing diagram illustrating a messaging sequence associated with a client computer **530**, message management module **116** and a match engine module **106**. Client computer **530** transmits electronic data transaction request message 1 to data transaction processing system **100** including message management module **116**. The message management module **116**, which implements TCP, generates an acknowledgement message ACK1 acknowledging receipt of electronic data transaction request message 1. The message management module **116** also generates a unique identification number ID1 associated with electronic data transaction request message 1. The message management module modifies the TCP ACK response message ACK1 to include ID1. Thus, the client computer receives ACK1 and ID1 before electronic data transaction request message 1 has been processed by the match engine module.

[0350] The match engine module processes electronic data transaction request message 1. The amount of time required to process electronic data transaction request message 1 may be highly variable. For example, the latency associated with the transaction processor/match engine module may be very high if the contents of electronic data transaction request message 1 result in the matching of many orders. After completing processing of electronic data transaction request message 1, the match engine module generates electronic data transaction result message 2 which contains information about the results of matching the transaction request in electronic data transaction request message 1. Electronic data transaction result message 2 includes ID1 associated with electronic data transaction request message 1. Match engine module transmits electronic data transaction result message 2 and ID1 to match engine module, which is transmitted by message

management module **116** to the client computer **530** associated with electronic data transaction request message 1.

[0351] In one embodiment, client computer **530** may be able to submit another electronic data transaction request message 3 to the data transaction processing system **100** before match engine module **106** has completed processing electronic data transaction request message 1. For example, as shown in FIG. **10**, after receiving the acknowledgement ACK1 which includes the identification number ID1 of electronic data transaction request message 1, but before the electronic data transaction request message 1 has been processed by the match engine module, the client computer **530** transmits another message, electronic data transaction request message 3, to the data transaction processing system. Electronic data transaction request message 3 includes the identification number ID1 for message 1. Thus, client computer **530** is able to reference message 1 in message 3 before message 1 has even been processed and reported on by the data transaction processing system.

[0352] For example, client computer **530** may want to modify the quantity associated with message 1, or may want to cancel message 1. If client computer **530** is not immediately provided the identification number for message 1 in the TCP level ACK response for message 1, client computer **530** must wait until match engine module has processed message 1. The TCP level ACK response is sent substantially immediately, so ID1 must be generated quickly as well. If generation of ID1 delays transmission of ACK1, the client computer **530**, per TCP, will transmit a duplicate of message 1 back to the data transaction processing system, unnecessarily burdening the communications infrastructure and adding to message management module's receipt/processing workload.

[0353] Because client computer **530** receives ID1 referencing the order/transaction request associated with message 1 in near real time, the client computer **530** can exhibit greater control over the contents and processing of message 1. In particular, client computer **530** can transmit message 3 which may modify message 1. Message 3 references message 1 via ID1. The message management module **116** may not need to generate a new identification number for message 3. Message management module **116** still generates an acknowledgement ACK3 that electronic data transaction request message 3 was received and transmits the ACK3 to client computer **530**. Message management module **116** may transmit message 3 and ID1 to match engine module **106** for processing.

[0354] Match engine module **106** may begin processing message 3 after message 1 is processed, as shown in FIG. **10**. If all of the quantity associated with message 1 matches, message 3 may be ineffective to control message 1. However, if some of the quantity associated with message 1 does not match and rests on the book associated with order book module **110**, message 3 may be effective to cancel that resting quantity. Thus, the effects of message 3, which references message 1 by using ID1, may depend on whether message 1 has already been processed by the match engine module **106**, and how the processing of message 1 impacted the order book associated with order book module **110**.

[0355] The illustrations of the embodiments described herein are intended to provide a general understanding of the structure of the various embodiments. The illustrations are not intended to serve as a complete description of all of the elements and features of apparatus and systems that utilize the structures or methods described herein. Many other embodiments may be apparent to those of skill in the art upon reviewing the disclosure. Other embodiments may be utilized and derived from the disclosure, such that structural and logical substitutions and changes may be made without departing from the scope of the disclosure. Additionally, the illustrations are merely representational and may not be drawn to scale. Certain proportions within the illustrations may be exaggerated, while other proportions may be minimized. Accordingly, the disclosure and the figures are to be regarded as illustrative rather than restrictive.

[0356] While this specification contains many specifics, these should not be construed as

limitations on the scope of the invention or of what may be claimed, but rather as descriptions of features specific to particular embodiments of the invention. Certain features that are described in this specification in the context of separate embodiments can also be implemented in combination in a single embodiment. Conversely, various features that are described in the context of a single embodiment can also be implemented in multiple embodiments separately or in any suitable sub-combination. Moreover, although features may be described as acting in certain combinations and even initially claimed as such, one or more features from a claimed combination can in some cases be excised from the combination, and the claimed combination may be directed to a sub-combination or variation of a sub-combination.

[0357] Similarly, while operations are depicted in the drawings and described herein in a particular order, this should not be understood as requiring that such operations be performed in the particular order shown or in sequential order, or that all illustrated operations be performed, to achieve desirable results. In certain circumstances, multitasking and parallel processing may be advantageous. Moreover, the separation of various system components in the described embodiments should not be understood as requiring such separation in all embodiments, and it should be understood that the described program components and systems can generally be integrated together in a single software product or packaged into multiple software products.

[0358] One or more embodiments of the disclosure may be referred to herein, individually and/or collectively, by the term “invention” merely for convenience and without intending to voluntarily limit the scope of this application to any particular invention or inventive concept. Moreover, although specific embodiments have been illustrated and described herein, it should be appreciated that any subsequent arrangement designed to achieve the same or similar purpose may be substituted for the specific embodiments shown. This disclosure is intended to cover any and all subsequent adaptations or variations of various embodiments. Combinations of the above embodiments, and other embodiments not specifically described herein, will be apparent to those of skill in the art upon reviewing the description.

[0359] The Abstract of the Disclosure is provided to comply with 37 C.F.R. § 1.72 (b) and is submitted with the understanding that it will not be used to interpret or limit the scope or meaning of the claims. In addition, in the foregoing Detailed Description, various features may be grouped together or described in a single embodiment for the purpose of streamlining the disclosure. This disclosure is not to be interpreted as reflecting an intention that the claimed embodiments require more features than are expressly recited in each claim. Rather, as the following claims reflect, inventive subject matter may be directed to less than all of the features of any of the disclosed embodiments. Thus, the following claims are incorporated into the Detailed Description, with each claim standing on its own as defining separately claimed subject matter.

[0360] It is therefore intended that the foregoing detailed description be regarded as illustrative rather than limiting, and that it be understood that it is the following claims, including all equivalents, that are intended to define the spirit and scope of this invention.

Claims

1. A computer implemented method comprising: receiving, by a network interface of a data transaction processing system, from a client computer remote from the network interface, via a network coupled therebetween, an electronic data transaction request message; generating, by the network interface, a unique identification number; augmenting, by the network interface, the electronic data transaction request message with the generated unique identification number; transmitting, by the network interface, the augmented electronic data transaction request message to a hardware matching processor for processing thereby, the hardware matching processor operative to process the augmented electronic data transaction request message, generate a data transaction result message comprising a result thereof, and communicate the data transaction result

message to the network interface for transmission to the client computer from which the electronic data request message was received; substantially immediately, in response to receipt of the electronic data transaction request message, before the electronic data transaction request message is processed by the hardware matching processor, augmenting, by the network interface, a transport layer acknowledgement message, generated based on a communications protocol, acknowledging receipt of the electronic data transaction request message to include the generated unique identification number; and communicating, by the network interface, the augmented transport layer acknowledgement message to the client computer, from which the electronic data transaction request message was received, prior to transmission of the data transaction result message comprising the result of the processing thereof by the hardware matching processor.

2. The computer implemented method of claim 1, further comprising: generating, by the network interface, the unique identification number based on a monotonically increasing counter.

3. The computer implemented method of claim 1, further comprising: generating, by the network interface, the unique identification number based on a hardware generated timestamp data associated with the electronic data transaction request message.

4. The computer implemented method of claim 3, further comprising: generating, by a clock, the hardware timestamp data associated with the electronic data transaction request message.

5. The computer implemented method of claim 3, wherein the hardware timestamp data includes data indicative of a timestamp denoting a time at which the electronic data transaction request message was received by the network interface.

6. The computer implemented method of claim 3, further comprising: adding, by the network interface, the hardware timestamp data to an optional data header of the transport layer acknowledgement message.

7. The computer implemented method of claim 1, wherein the electronic data transaction request message comprises a format for transmission using a Transmission Control Protocol (TCP).

8. The computer implemented method of claim 1, wherein the processing includes attempting to match, by the hardware matching processor, the electronic data transaction request message with at least one of previously received but unsatisfied electronic data transaction request messages for a transaction which is counter thereto.

9. The computer implemented method of claim 1, wherein the data transaction result message includes the generated unique identification number of the processed augmented electronic data transaction request message.

10. The computer implemented method of claim 1, wherein the electronic data transaction request message includes an instruction to perform a transaction on a data object, a quantity, and a value.

11. A data transaction processing system comprising: a network interface configured to: receive, from a client computer remote from the network interface, via a network coupled therebetween, an electronic data transaction request message; generate a unique identification number; augment the electronic data transaction request message with the generated unique identification number; transmit the augmented electronic data transaction request message to a hardware matching processor for processing thereby, the hardware matching processor operative to process the augmented electronic data transaction request message, generate a data transaction result message comprising a result thereof, and communicate the data transaction result message to the network interface for transmission to the client computer from which the electronic data request message was received; substantially immediately, in response to receipt of the electronic data transaction request message, before the electronic data transaction request message is processed by the hardware matching processor, augment a transport layer acknowledgement message, generated based on a communications protocol, acknowledging receipt of the electronic data transaction request message to include the generated unique identification number; and communicate the augmented transport layer acknowledgement message to the client computer, from which the electronic data transaction request message was received, prior to transmission of the data

transaction result message comprising the result of the processing thereof by the hardware matching processor.

12. The data transaction processing system of claim 11, wherein the network interface is configured to generate, by the network interface, the unique identification number based on a monotonically increasing counter.

13. The data transaction processing system of claim 11, wherein the network interface is configured to generate the unique identification number based on a hardware generated timestamp data associated with the electronic data transaction request message.

14. The data transaction processing system of claim 13, further comprising: a clock configured to generate the hardware generated timestamp data.

15. The data transaction processing system of claim 13, wherein the hardware timestamp data includes data indicative of a timestamp denoting a time at which the electronic data transaction request message was received by the network interface.

16. The data transaction processing system of claim 13, wherein the hardware timestamp data is added to an optional data header of the transport layer acknowledgement message.

17. The data transaction processing system of claim 11, wherein the electronic data transaction request message comprises a format for transmission using a Transmission Control Protocol (TCP).

18. The data transaction processing system of claim 11, wherein the processing by the hardware matching processor includes attempting to match the electronic data transaction request message with at least one previously received but unsatisfied electronic data transaction request message for a transaction which is counter thereto.

19. The data transaction processing system of claim 11, wherein the data transaction result message includes the generated unique identification number.

20. The data transaction processing system of claim 11, wherein the electronic data transaction request message includes an instruction to perform a transaction on a data object, a quantity, and a value.

21. A system comprising: means for receiving, from a client computer, via a network coupled therebetween, an electronic data transaction request message; means for generating a unique identification number; means for augmenting the electronic data transaction request message with the generated unique identification number; means for transmitting the augmented electronic data transaction request message to a hardware matching processor for processing thereby, the hardware matching processor operative to process the augmented electronic data transaction request message, generate a data transaction result message comprising a result thereof, and communicate the data transaction result message to the network interface for transmission to the client computer from which the electronic data request message was received; substantially immediately, in response to receipt of the electronic data transaction request message, before the electronic data transaction request message is processed by the hardware matching processor, means for augmenting a transport layer acknowledgement message, generated based on a communications protocol, acknowledging receipt of the electronic data transaction request message to include the generated unique identification number; and means for communicating the augmented transport layer acknowledgement message to the client computer, from which the electronic data transaction request message was received, prior to transmission of the data transaction result message comprising the result of the processing thereof by the hardware matching processor.